

**Department of Robotics and Automation**

**Computer Vision for Robotics**

**CA-3: Vision Based Application Task**

**"CV Autonomous Navigation: Semantic  
Perception and Topological Memory on ROS 2  
Nav2"**

**By**

**Zubin Patnaik (24070127142)**

**Utkarsh Singh (24070127128)**

**Shubham Desai (24070127114)**

**Soham Darade (24070127118)**



**Under the Guidance of**

**Dr. Sameer Sayyad**

Assistant Professor,

Department of Robotics and Automation,  
Symbiosis Institute of Technology, Pune

**SYMBIOSIS INSTITUTE OF TECHNOLOGY**

**A Constituent of Symbiosis International (Deemed University),**

**Pune-412115**

**2026-27**

# Contents

|          |                                                                   |           |
|----------|-------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                               | <b>1</b>  |
| 1.1      | Background/Rationale . . . . .                                    | 1         |
| 1.2      | Motivation . . . . .                                              | 1         |
| 1.3      | Recent Trends . . . . .                                           | 2         |
| 1.4      | Significance of the Study . . . . .                               | 2         |
| 1.5      | Organization of the Report . . . . .                              | 2         |
| <b>2</b> | <b>Review of Relevant Literature</b>                              | <b>3</b>  |
| 2.1      | Previous Research Work . . . . .                                  | 3         |
| 2.1.1    | Robot Middleware, Navigation, and Simulation . . . . .            | 3         |
| 2.1.2    | Real-Time Object Detection . . . . .                              | 4         |
| 2.1.3    | Synthetic Training Data and Simulation-to-Real Transfer . . . . . | 4         |
| 2.1.4    | Template Matching for Sign Recognition . . . . .                  | 4         |
| 2.1.5    | Camera–LiDAR Fusion . . . . .                                     | 4         |
| 2.1.6    | Semantic and Topological Mapping . . . . .                        | 5         |
| 2.2      | Key Findings . . . . .                                            | 6         |
| 2.3      | Research Gaps . . . . .                                           | 7         |
| <b>3</b> | <b>Problem Statement and Objectives</b>                           | <b>8</b>  |
| 3.1      | Research Questions . . . . .                                      | 8         |
| 3.2      | Statement of the Problem . . . . .                                | 8         |
| 3.3      | Objectives . . . . .                                              | 8         |
| 3.4      | Scope and Limitations . . . . .                                   | 13        |
| <b>4</b> | <b>Methodology</b>                                                | <b>14</b> |
| 4.1      | Overview of Methodology . . . . .                                 | 14        |
| 4.1.1    | Design Decisions . . . . .                                        | 15        |
| 4.2      | Objective-wise Methodology . . . . .                              | 16        |
| 4.2.1    | Object Detection (Objectives 2, 3 and 12) . . . . .               | 16        |
| 4.2.2    | Directional Sign Perception (Objective 6) . . . . .               | 17        |

|          |                                                                          |           |
|----------|--------------------------------------------------------------------------|-----------|
| 4.2.3    | LiDAR-Camera Sensor Fusion (Objective 4) . . . . .                       | 18        |
| 4.2.4    | Object Tracking and Time-to-Collision (Objective 5) . . . . .            | 18        |
| 4.2.5    | Topological Memory and Decision Engine (Objectives 7, 8 and 9) . . . . . | 19        |
| 4.2.6    | Monitoring Dashboard (Objective 11) . . . . .                            | 19        |
| 4.3      | Design . . . . .                                                         | 20        |
| 4.4      | Materials and Methods . . . . .                                          | 21        |
| 4.4.1    | Simulated Facility . . . . .                                             | 22        |
| 4.5      | Data Acquisition . . . . .                                               | 23        |
| 4.6      | Implementation . . . . .                                                 | 23        |
| <b>5</b> | <b>Results and Discussion</b>                                            | <b>25</b> |
| 5.1      | Presentation of Results . . . . .                                        | 25        |
| 5.1.1    | Detector Progression and the Data-Leakage Finding . . . . .              | 25        |
| 5.1.2    | YOLOv8n v2.5: Dataset and Training . . . . .                             | 26        |
| 5.1.3    | YOLOv8n v2.5: Detection Accuracy . . . . .                               | 26        |
| 5.1.4    | Robustness Across Distance, Angle, Lighting, Blur and Occlusion          | 28        |
| 5.1.5    | Controlled Perception Tests . . . . .                                    | 29        |
| 5.1.6    | Live ROS 2 Pipeline Checks . . . . .                                     | 30        |
| 5.1.7    | Directional Sign Perception . . . . .                                    | 30        |
| 5.1.8    | LiDAR-Camera Fusion Accuracy . . . . .                                   | 30        |
| 5.1.9    | Replanning Performance . . . . .                                         | 31        |
| 5.1.10   | Four-Scenario Test Suite . . . . .                                       | 31        |
| 5.1.11   | Ten-Mission Navigation Benchmark in Gazebo . . . . .                     | 32        |
| 5.1.12   | Ablation Study . . . . .                                                 | 33        |
| 5.1.13   | Full-Facility Traversal (Reported Only) . . . . .                        | 34        |
| 5.1.14   | Comparative Analysis . . . . .                                           | 34        |
| 5.1.15   | V2.6 Reliability Fixes . . . . .                                         | 34        |
| 5.1.16   | V2.6 Controlled Experiments . . . . .                                    | 35        |
| 5.1.17   | V2.6 Ten-Mission Benchmark . . . . .                                     | 37        |
| 5.1.18   | V3.0: Hospital Logistics Extension . . . . .                             | 38        |
| 5.2      | Discussion . . . . .                                                     | 41        |
| 5.2.1    | Interpretation of Results . . . . .                                      | 41        |
| 5.2.2    | Significance of Findings . . . . .                                       | 42        |
| 5.2.3    | Limitations . . . . .                                                    | 42        |
| 5.3      | Summary of Results . . . . .                                             | 44        |

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>6</b> | <b>Work to be Done</b>                                    | <b>46</b> |
| 6.1      | Conclusion . . . . .                                      | 46        |
| 6.2      | Work to be Done . . . . .                                 | 47        |
|          | <b>References</b>                                         | <b>49</b> |
|          | <b>Annexure I: Deployment Defects and Reproducibility</b> | <b>51</b> |
|          | <b>Annexure II: Simulated Facility Object Inventory</b>   | <b>53</b> |
|          | <b>Appendix A: Similarity Report</b>                      | <b>57</b> |

# List of Figures

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |    |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 4.1 | System architecture: from camera to detection, sign confirmation, LiDAR-camera fusion and time-to-collision, and the Decision Engine with Nav2. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                    | 14 |
| 4.2 | End-to-end perception and sensor-fusion pipeline (V2.6) . . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | 15 |
| 4.3 | The monitoring dashboard (Mission Monitor view) with the robot idle at the start node. Top: pipeline status and telemetry bar. Centre: the topological map with destinations and dead ends. Right: route-cost breakdown, multi-criteria route candidates and LiDAR clearances. Bottom: the annotated camera view with a <b>hospital_bed</b> and a <b>directional_sign</b> detection, and the visual-landmark panel showing a confirmed HOSPITAL sign (92% confidence, 6.8 m, direction right, target node <b>junction_1</b> ) with the nearest fused object and time to collision. . . . . | 20 |
| 5.1 | Normalized confusion matrix of YOLOv8n v2.5 on the validation split (columns: true class; rows: predicted class; the last row and column are the background). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                      | 27 |
| 5.2 | Precision–recall curves of YOLOv8n v2.5 on the validation split, per class and for all classes (mAP@0.50 = 0.922 as computed at the end of training). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                              | 28 |
| 5.3 | The robot in the main lobby of the V3.0 hospital, with the reception desk, visitors and two directional signs. . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 38 |
| 5.4 | Recorded paths of the fifteen V3.0 missions on the hospital map, from the collection point (circle) to the laboratory (square). . . . .                                                                                                                                                                                                                                                                                                                                                                                                                                                    | 40 |

# List of Tables

|      |                                                                                                                                                                                    |    |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 2.1  | Summary of the reviewed literature and its relevance to this project .                                                                                                             | 6  |
| 3.1  | Objectives and how the project addresses them . . . . .                                                                                                                            | 9  |
| 4.1  | Design decisions and rationale . . . . .                                                                                                                                           | 16 |
| 4.2  | Main Nav2 configuration ( <code>nav2_params.yaml</code> ; v2.5 value $\rightarrow$ V2.6 value where changed) . . . . .                                                             | 21 |
| 4.3  | Robot, sensors and environment . . . . .                                                                                                                                           | 22 |
| 4.4  | Dynamic agents in the simulated facility . . . . .                                                                                                                                 | 23 |
| 5.1  | Overall performance of YOLOv8n v2.5 on the leak-free validation and test splits . . . . .                                                                                          | 26 |
| 5.2  | Per-class results of YOLOv8n v2.5, with the v2.4 model on the same leak-free validation split for comparison . . . . .                                                             | 26 |
| 5.3  | mAP@0.50 on nine stress subsets (60 samples each) . . . . .                                                                                                                        | 29 |
| 5.4  | Controlled perception tests . . . . .                                                                                                                                              | 29 |
| 5.5  | Fusion range estimate against ground truth (synthetic scan, 1.5 cm noise) . . . . .                                                                                                | 31 |
| 5.6  | Block-and-replan timing on the 38-node graph (2,000 runs per case) .                                                                                                               | 31 |
| 5.7  | Four-scenario test suite results . . . . .                                                                                                                                         | 32 |
| 5.8  | Ten-mission benchmark with the v2.5 detector (from the project's V2.5 validation documentation; totals as recorded in <code>v24_mission_validation_results.json</code> ) . . . . . | 32 |
| 5.9  | Ten-mission benchmark: the two v2.4 runs against v2.5 (all values read directly from the results files) . . . . .                                                                  | 33 |
| 5.10 | Ablation of obstacle-handling modalities (2D simulation, 5 scenarios)                                                                                                              | 34 |
| 5.11 | Runtime TTC tests in Gazebo after the V2.6 fixes (two runs) . . . . .                                                                                                              | 36 |
| 5.12 | Inflation-radius sweep: distance travelled in 55 s (m), with minimum clearance (m) and replans in brackets . . . . .                                                               | 36 |
| 5.13 | V2.6 ten-mission benchmark (180 s limit per mission) . . . . .                                                                                                                     | 37 |

|      |                                                                                                          |    |
|------|----------------------------------------------------------------------------------------------------------|----|
| 5.14 | Detector on the V3 test split of Gazebo renders (500 images, 1,504 instances, confidence 0.35) . . . . . | 39 |
| 5.15 | Camera–LiDAR association on recorded Gazebo frames . . . . .                                             | 39 |
| 5.16 | V3.0 logistics missions in the hospital (collection point → laboratory)                                  | 40 |
| 5.17 | Technical limitations of v2.5 identified in the project’s audit . . . . .                                | 43 |

# Chapter 1

## Introduction

This chapter introduces the problem of semantic indoor navigation, explains the motivation for the project, and outlines the structure of the rest of the report.

### 1.1 Background/Rationale

Indoor mobile robots operating in structured environments such as hospitals, warehouses, and laboratories face a problem that classical metric navigation alone cannot solve: metric occupancy maps encode geometry, not meaning. A robot navigating by costmap alone has no notion of which corridor leads to which department, cannot read a signboard telling it to turn, and will repeatedly attempt to enter a cul-de-sac unless that geometry happens to be marked impassable.

Real facilities also change continuously. Misplaced carts, temporarily parked equipment, and foot traffic obstruct designated routes in ways a static map cannot anticipate.

### 1.2 Motivation

This project addresses that gap by layering semantic perception and cognitive memory on top of a standard ROS 2 Nav2 stack. Classical concerns (metric costmaps, path planning, and trajectory execution) remain strictly delegated to Nav2's NavFn planner and Regulated Pure Pursuit controller. A custom Decision Engine sits above this layer and is responsible for everything Nav2 cannot do on its own:

- interpreting directional signage,
- remembering the facility's topology across restarts, and
- rerouting around corridors that become impassable at runtime.

## 1.3 Recent Trends

The relevant trends are as follows:

- Lightweight one-stage object detectors (the YOLO family) that run in real time (Redmon et al., 2016; Terven and Córdova-Esparza, 2023).
- Fusion of camera semantics with LiDAR range data (Vora et al., 2020).
- Semantic and topological maps that give robots more than geometry (Kostavelis and Gasteratos, 2015; Garcia-Fidalgo and Ortiz, 2015).
- Training perception models on simulated images and transferring them to real cameras (Tobin et al., 2017).
- ROS 2 and Nav2 as the common software base for mobile robot navigation (Macenski et al., 2022, 2020).

## 1.4 Significance of the Study

The project shows that a semantic layer can sit on top of Nav2 without compromising it: Nav2 keeps full responsibility for collision-free motion, while a SQLite-backed topological graph adds destination-by-name routing, persistent memory, dead-end avoidance, and dynamic replanning. The report also separates verified results from results that come only from offline or synthetic tests (Chapter 5), so the strength of each claim is stated explicitly.

## 1.5 Organization of the Report

Chapter 2 reviews relevant literature. Chapter 3 states the problem, objectives, and scope. Chapter 4 describes the system architecture, the perception pipeline, the topological memory, and the implementation. Chapter 5 presents and discusses results. Chapter 6 covers conclusions and the work still to be done. Deployment defects found while reproducing the project on a second machine are documented in Annexure I.

# Chapter 2

## Review of Relevant Literature

This chapter reviews the work on which the project builds, grouped by the component it supports: the navigation stack, object detection, synthetic training data, sign recognition, camera–LiDAR fusion, and semantic and topological mapping. Table 2.1 summarizes the works and their relevance.

### 2.1 Previous Research Work

#### 2.1.1 Robot Middleware, Navigation, and Simulation

ROS 2 was redesigned from the ground up for modern robotic systems, and its adoption has accelerated the deployment of real robots in varied environments (Macenski et al., 2022). Its navigation stack, Nav2, uses a behavior tree to orchestrate navigation tasks and employs methods designed for dynamic environments and a wide range of sensors; it was demonstrated operating safely alongside students on a campus over a marathon-length run (Macenski et al., 2020). Trajectory execution in Nav2 can use Regulated Pure Pursuit, a Pure Pursuit variant that adds heuristics to regulate linear velocity, with a focus on safety in constrained and partially observable spaces (Macenski et al., 2023). Gazebo is the open-source 3D multi-robot simulator used for testing, originally designed to recreate the complex worlds that mobile robots encounter (Koenig and Howard, 2004).

For graph search, A\* incorporates heuristic information into a formal theory of graph searching and has an optimality property for a class of search strategies (Hart et al., 1968). In this project the Nav2 global planner (NavFn) is configured to use A\*, and the topological layer searches its own cost-weighted graph for the best routes.

### **2.1.2 Real-Time Object Detection**

YOLO framed object detection as a regression problem: a single network predicts bounding boxes and class probabilities directly from full images in one evaluation, which allows real-time processing (Redmon et al., 2016). The family has since evolved through many versions up to YOLOv8, as surveyed by Terven and Córdova-Esparza (2023), who also note its importance for robotics applications. This project uses the nano variant of YOLOv8 (Jocher et al., 2023), chosen for its low inference time and memory footprint on a laptop GPU.

### **2.1.3 Synthetic Training Data and Simulation-to-Real Transfer**

Domain randomization trains models on simulated images with randomized rendering so that the real world appears to the model as one more variation; it has been used to train a real-world object detector, accurate to 1.5 cm, using only simulated images with non-realistic random textures (Tobin et al., 2017). This is relevant to the project’s v1 detector, which is trained on script-drawn images with randomized backgrounds, and to its v2.4 detector, which is fine-tuned on Gazebo images. It also frames the central open question of this work: how well accuracy measured on synthetic images carries over to the robot’s live camera view.

### **2.1.4 Template Matching for Sign Recognition**

Normalized cross-correlation is the standard similarity measure for template matching. Lewis (1995) showed that it can be computed efficiently by normalizing the cross-correlation with precomputed tables of integrals of the image. The project’s sign reader uses this measure against a closed set of 15 known sign textures, which makes it deterministic and needs no training data.

### **2.1.5 Camera–LiDAR Fusion**

Camera and LiDAR are complementary: images carry semantic detail while LiDAR carries precise range. PointPainting is a sequential fusion method that projects LiDAR points into the output of an image-only segmentation network and appends the class scores to each point, so that any LiDAR-only method can use them (Vora et al., 2020). The project’s fusion node follows the same idea at a much smaller scale: it maps a 2D bounding box to an angular sector of a 2D LiDAR scan and takes the

median range in that sector.

### **2.1.6 Semantic and Topological Mapping**

Semantic mapping adds meaning to the geometry a robot builds; Kostavelis and Gasteratos (2015) survey the techniques for including semantic information in navigation systems, divided into human-assisted and autonomous approaches. Topological maps model the environment as a graph in which nodes are distinctive places and edges are the relationships between them, an alternative to metric maps that is simple and cheap to store (Garcia-Fidalgo and Ortiz, 2015). Metric and semantic representations for long-term mapping are also covered in the wider SLAM survey by Cadena et al. (2016).

**Table 2.1** Summary of the reviewed literature and its relevance to this project

| Reference                                               | Contribution                                         | Use in this project                     |
|---------------------------------------------------------|------------------------------------------------------|-----------------------------------------|
| Macenski et al. (2022)                                  | ROS 2 design, architecture, and case studies         | Middleware for all nodes                |
| Macenski et al. (2020)                                  | Nav2 navigation system with behavior trees           | Metric planning and control             |
| Macenski et al. (2023)                                  | Regulated Pure Pursuit path tracking                 | Trajectory controller                   |
| Koenig and Howard (2004)                                | Gazebo 3D multi-robot simulator                      | Simulation environment                  |
| Hart et al. (1968)                                      | A* heuristic graph search                            | Route planning on the topological graph |
| Redmon et al. (2016), Terven and Córdova-Esparza (2023) | One-stage real-time detection; YOLO survey           | Obstacle detector                       |
| Tobin et al. (2017)                                     | Domain randomization for simulation-to-real transfer | Synthetic dataset design and its limits |
| Lewis (1995)                                            | Fast normalized cross-correlation                    | Sign template matching                  |
| Vora et al. (2020)                                      | Sequential camera-LiDAR fusion                       | Basis of the fusion idea                |
| Kostavelis and Gasteratos (2015)                        | Survey of semantic mapping                           | Motivation for the semantic layer       |
| Garcia-Fidalgo and Ortiz (2015)                         | Survey of topological mapping                        | Graph-based facility memory             |
| Cadena et al. (2016)                                    | SLAM survey, including semantic maps                 | Context for map representations         |

## 2.2 Key Findings

- Nav2 provides collision-free metric planning and control, but it works on costmaps and does not itself represent semantic meaning (Macenski et al., 2020).
- Real-time one-stage detectors such as YOLO are well suited to robotics, and the nano variants fit laptop-class GPUs (Redmon et al., 2016; Terven and Córdova-Esparza, 2023).

- Training on simulated or randomized images can produce detectors that work on real images, but this depends on how much variation the simulator provides (Tobin et al., 2017).
- Semantic information and topological graphs are established ways of giving robots more than geometric maps (Kostavelis and Gasteratos, 2015; Garcia-Fidalgo and Ortiz, 2015).
- Class information from images can be attached to LiDAR range data to improve what the range data alone can support (Vora et al., 2020).

## 2.3 Research Gaps

1. Nav2’s costmaps carry geometry only. The stack has no built-in way to read directional signage, remember which corridor leads where across restarts, or avoid a known dead end unless its geometry is marked impassable.
2. The surveys reviewed here (Kostavelis and Gasteratos, 2015; Garcia-Fidalgo and Ortiz, 2015) describe semantic and topological representations in general. This project’s contribution is integrating such a layer above Nav2, without modifying Nav2’s own planner and controller: the Decision Engine sits above Nav2 instead of inside it.
3. Accuracy measured on synthetic images is known to depend on how well the simulator covers the real distribution (Tobin et al., 2017). This project met the problem in practice: its v2.4 detector scored 0.910 on a test split that shared near-duplicate scenes with its training data, and 0.634 once the data was rebuilt without leakage.

# Chapter 3

## Problem Statement and Objectives

### 3.1 Research Questions

1. Can a semantic and topological layer be added above Nav2 without weakening its collision-free control?
2. Can the robot read directional signage and route by destination name rather than by coordinates?
3. Can the robot remember a facility’s topology across restarts, avoid dead ends, and reroute around blocked corridors within a 1 ms planning budget?

### 3.2 Statement of the Problem

Metric occupancy maps encode geometry, not meaning. A robot that navigates by costmap alone cannot tell which corridor leads to which department, cannot read a signboard, and will repeatedly enter a dead end unless its geometry is marked impassable. Facilities also change at runtime, so a static map cannot anticipate blocked routes.

### 3.3 Objectives

The project has fourteen objectives. Table 3.1 states each objective and how the project addresses it, together with an honest status: *achieved*, *partly achieved*, or *not achieved*. Where an objective is only partly met, the reason is given and the supporting evidence is in Chapter 5.

**Table 3.1** Objectives and how the project addresses them

| No. | Objective                                                                                                                                                                  | How the project addresses it, and status                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|-----|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | To develop an autonomous mobile robot navigation system capable of navigating a realistic indoor environment between predefined destinations using ROS 2, Gazebo and Nav2. | A ROS 2 Lyrical / Gazebo Sim / Nav2 stack runs in a $32 \times 26$ m facility with 11 named destinations; the Decision Engine turns a destination name into topological waypoints and sends them to Nav2 (Sections 4.2.5 and 4.3). <b>Partly achieved:</b> the robot travels 230.84 m with no collisions, but none of the 10 benchmark missions reaches its goal in time (0/10; the final v2.4 run completed 2/10), Section 5.1.11. After the V2.6 fixes, 5/10 missions reach their goal with 0 collisions (Section 5.1.17). In the V3.0 hospital, 10/10 logistics missions without traffic and 5/5 with traffic are completed on one repeated route (Section 5.1.18). |
| 2   | To implement computer-vision-based object detection using YOLOv8 for identifying relevant environmental objects and directional signs.                                     | A YOLOv8n detector fine-tuned on a leak-free 2,000-image dataset detects 10 classes including <code>directional_sign</code> (Section 4.2.1). <b>Achieved:</b> 0.899 mAP@0.50 on both validation and test splits (Section 5.1.2; confusion matrix and precision–recall curve in Figs. 5.1 and 5.2); on Gazebo renders v2.5 reaches only 0.106, and the V3 model 0.804 (Section 5.1.18); recall on carts, forklifts and signs is weaker in the controlled tests (Section 5.1.5).                                                                                                                                                                                         |
| 3   | To develop a reliable real-time perception pipeline incorporating configurable detection thresholds, temporal confirmation, frame skipping and GPU-accelerated inference.  | Thresholds and box filters are set per class in <code>perception_v25.yaml</code> ; signs pass a temporal confirmation gate; every second frame is processed; inference runs on <code>cuda:0</code> (Sections 4.2.1 and 4.2.2; pipeline in Fig. 4.2). <b>Achieved:</b> 4.06 ms per image (246 FPS) and about 175 MB of GPU memory (Section 5.1.6).                                                                                                                                                                                                                                                                                                                      |

| No. | Objective                                                                                                                               | How the project addresses it, and status                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 4   | To perform LiDAR–camera sensor fusion for estimating the position and distance of detected objects within the robot’s environment.      | Laser points are clustered and associated with the camera ray of each detection, giving a labelled obstacle position in the <code>base_link</code> frame (Section 4.2.3). <b>Achieved</b> at algorithm level: error under 1.3 cm on synthetic scans (Section 5.1.8); a V2.6 live capture shows the node running but records no fused obstacle, so accuracy on live Gazebo returns is still not measured (Section 5.1.16). On recorded Gazebo frames, V3.0 measures 51.4% correct association for the V2.6 method and 80.6% for its own (Section 5.1.18). |
| 5   | To implement dynamic-obstacle detection and collision-risk estimation using object tracking and Time-to-Collision (TTC).                | A centroid tracker with exponential smoothing follows fused obstacles, and TTC is computed from the radial closing rate; the robot yields when TTC falls below 1.8 s (Section 4.2.4). <b>Partly achieved:</b> no collisions occurred, but in v2.5 TTC ignores the robot’s own speed and is applied to static objects, causing 95 spurious yields (Section 5.2.3). V2.6 adds ego-motion compensation and a dynamic-object guard; its runtime tests pass 4/4 in one run and 3/4 in the next, with one false yield (Table 5.11).                            |
| 6   | To develop semantic sign recognition and navigation by associating detected directional signs with destinations through a semantic map. | Confirmed signs are matched to the 16 signs in <code>semantic_map.yaml</code> by expected bearing and range, and a “closed” sign raises the cost of the edge it blocks (Sections 4.2.2 and 4.2.5). <b>Partly achieved:</b> 29 sign observations were used across the benchmark, but overall sign recall in the controlled test is 58% (Section 5.1.5).                                                                                                                                                                                                   |

| No. | Objective                                                                                                                                                                                | How the project addresses it, and status                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7   | To implement topological and semantic route planning using a graph representation of the facility and route selection based on available navigation paths.                               | The facility is a 53-node graph; routes are chosen from the three lowest-cost alternatives using a multi-criteria edge cost (Section 4.2.5). <b>Achieved:</b> all 7 tested destinations are reached with 0 dead-end entries (Section 5.1.10).                                                                                                                                                                                    |
| 8   | To implement dynamic replanning and obstacle avoidance to enable the robot to respond to changing obstacles, blocked paths, dead ends and navigation failures.                           | Blocked edges are penalized and the route is replanned, with Nav2 handling local avoidance (Sections 4.2.5 and 4.3). <b>Partly achieved:</b> the search itself takes 0.05 ms median (Section 5.1.9), but in the v2.5 Gazebo benchmark the robot replans 327 times and gets caught in replan cascades in narrow corridors (Section 5.1.11). A V2.6 live test reroutes around a blocked corridor with no cascade (Section 5.1.16). |
| 9   | To develop a persistent navigation memory system using SQLite for storing and retrieving navigation and topological edge information.                                                    | A SQLite database (WAL mode) stores nodes, edges, route segments, journeys, traversals, obstacles, recovery events and sign detections (Section 4.2.5). <b>Achieved</b> for persistence (scenario 4 passes); <b>defect:</b> temporary blockages stay marked across runs (30 edges) and distort later routes (Section 5.2.3); V2.6 resets them at the start of each mission, but the effect is not separately measured.           |
| 10  | To integrate the complete autonomous navigation stack comprising perception, sensor fusion, semantic understanding, decision-making, route planning, Nav2 control and Gazebo simulation. | All components run together in one containerized ROS 2 system (Section 4.6) and were run through ten full missions. <b>Achieved</b> as integration; end-to-end performance is limited by the defects in objectives 5, 8 and 9.                                                                                                                                                                                                   |

| No. | Objective                                                                                                                                                                                                                                            | How the project addresses it, and status                                                                                                                                                                                                                      |
|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 11  | To develop a real-time monitoring and visualization system for displaying robot state, perception, detected objects, semantic signs, navigation status and TTC telemetry.                                                                            | A web dashboard shows the annotated camera view, detections, the visual-landmark panel, the topological map, TTC and GPU telemetry (Section 4.2.6 and Fig. 4.3). <b>Achieved:</b> the landmark panel was debugged and now clears stale signs after 2.0 s.     |
| 12  | To optimize the perception system for real-time operation by improving detection accuracy while maintaining practical inference latency and GPU/VRAM utilization.                                                                                    | The v2.5 dataset and training raised mAP@0.50 from 0.634 to 0.899 on the same leak-free split while inference stayed at about 4 ms per image (3.94 ms for v2.4, 4.06 ms for v2.5) and GPU memory at about 175 MB (Section 5.1.14). <b>Achieved.</b>           |
| 13  | To experimentally validate the integrated autonomous navigation system using controlled tests and multi-mission benchmarks involving navigation success, distance, completion time, clearance, replanning, TTC events, collisions and interventions. | Eight controlled perception tests, a ten-mission benchmark and the V2.6 experiments record each of these quantities (Sections 5.1.5, 5.1.11 and 5.1.16). <b>Achieved:</b> the validation was carried out and reported in full, including the negative result. |
| 14  | To identify and diagnose system limitations through experimental analysis of perception, TTC behavior, route blockage, corridor feasibility, velocity regulation and waypoint navigation, providing a basis for subsequent V2.6 improvements.        | Nine limitations are documented with evidence and root cause (Table 5.17) and led directly to the V2.6 fixes (Section 5.1.15). <b>Achieved.</b>                                                                                                               |

### 3.4 Scope and Limitations

The work is carried out entirely in Gazebo simulation on the ROS 2 Lyrical / Nav2 stack. The main limitations are:

- Only half of the benchmark missions are completed (V2.6, 5 of 10; v2.5, 0 of 10). The causes of the v2.5 result are software defects and tuning, listed in Table 5.17; V2.6 fixes them in the code, but four missions still time out and one stays stuck.
- The detector is trained and validated on script-generated images (2,000 in the current version); only the ten-mission benchmark exercises the live Gazebo camera.
- Directional-sign detection and cart and forklift recall are weak in the controlled tests (58%, 73% and 77%).
- Sign and fusion accuracy also come partly from constructed inputs, not live sensor data.
- The topology graph is hand-authored for each facility.
- No result has been checked on physical hardware.

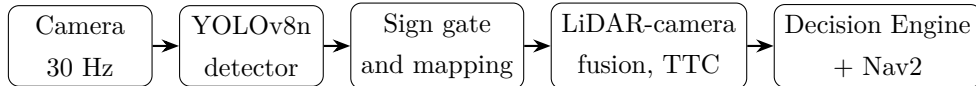
These are discussed further in Chapter 5.

# Chapter 4

## Methodology

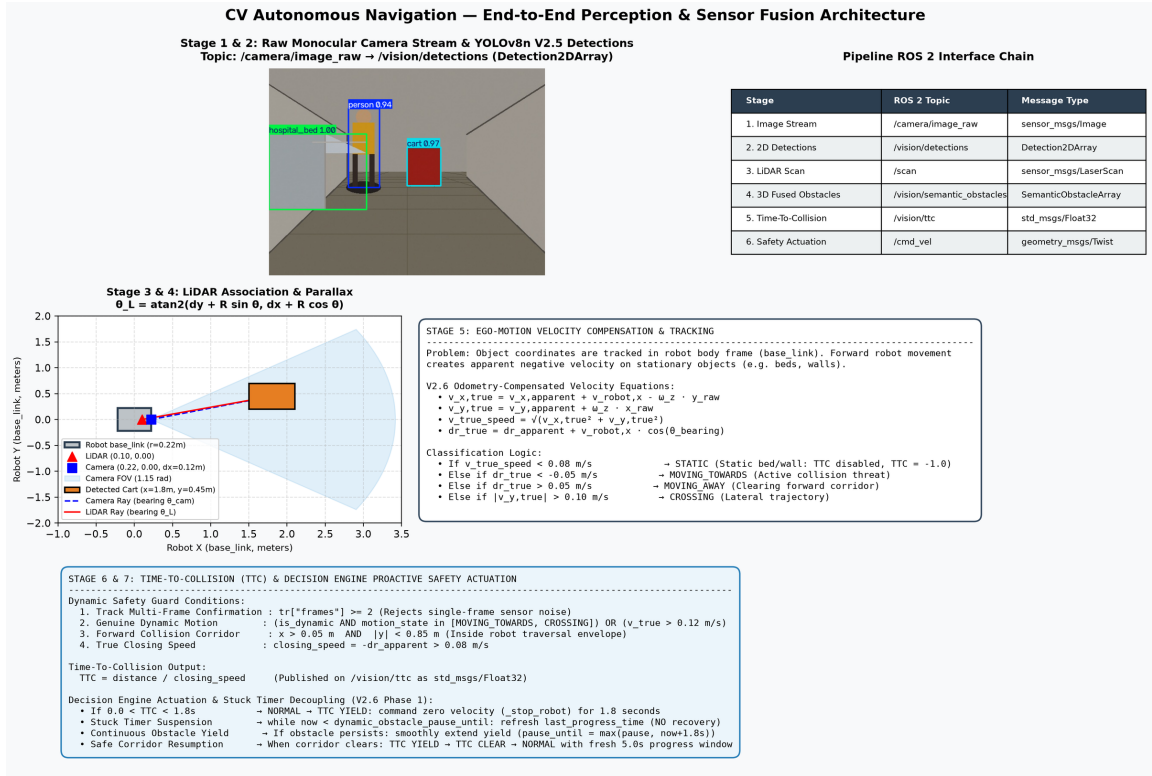
### 4.1 Overview of Methodology

The system is a pipeline built on ROS 2 Nav2 and simulated in Gazebo (Fig. 4.1). Each layer keeps a single responsibility. The perception node (`/object_detection_node`) only observes; the fusion node converts observations into metric obstacles, tracks them, and estimates time to collision; the Decision Engine is the only component that reasons about topology, signs, and safety yields and dispatches Nav2 goals; Nav2 itself never sees semantic meaning, only costmaps and an action goal. This separation keeps Nav2’s collision-free trajectory tracking intact while adding a cognitive layer on top of it rather than inside it.



**Fig. 4.1.** System architecture: from camera to detection, sign confirmation, LiDAR-camera fusion and time-to-collision, and the Decision Engine with Nav2.

Fig. 4.2 shows the same pipeline in more detail, with the ROS 2 topic and message type at each stage and the V2.6 ego-motion compensation and yield logic.



**Fig. 4.2.** End-to-end perception and sensor-fusion pipeline (V2.6): camera stream and YOLOv8n detections, LiDAR association, ego-motion-compensated tracking, and time-to-collision with the Decision Engine’s safety yield. The camera view is a sample frame; the other panels are schematic.

### 4.1.1 Design Decisions

Table 4.1 lists the main design choices and the alternatives that were rejected.

**Table 4.1** Design decisions and rationale

| Decision                                                             | Alternative                                | Why this choice                                                                                                                                                                                       |
|----------------------------------------------------------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YOLOv8n (Nano) detector                                              | Larger YOLOv8 variants                     | 4.06 ms per image and about 175 MB of GPU memory in the running system leave most of a 4 GB laptop GPU free, well inside the 30 FPS camera budget; YOLOv8s was 2.4 times slower with no accuracy gain |
| Signs matched to a known semantic map, confirmed over several frames | A learned sign classifier                  | The 16 signs are known in advance and registered with position and destination, so a detection is matched to a known landmark; a temporal gate removes single-frame errors                            |
| Penalize dead ends and blockages                                     | Delete those edges from the graph          | The graph stays connected, so a route always exists, and unblocking an edge restores the original path                                                                                                |
| Penalty ratio 100,000 : 1,000,000                                    | One shared penalty                         | Both dwarf any real path in a $32 \times 26$ m facility, and a known blockage ranks worse than a dead end, which might still be passable                                                              |
| SQLite for memory                                                    | In-process data structures                 | Signs, traversals, and blockages persist across restarts in a single file with no server                                                                                                              |
| Semantics above Nav2, not inside it                                  | Custom costmap plugins or a custom planner | Nav2 keeps its tested collision-free control; the semantic layer only chooses goals and adds costmap points                                                                                           |

## 4.2 Objective-wise Methodology

### 4.2.1 Object Detection (Objectives 2, 3 and 12)

The obstacle detector is a YOLOv8n model (Nano, about 3M parameters) fine-tuned for this project and run on an NVIDIA RTX 3050 Laptop GPU (cuda:0). Three versions were trained:

- **v1:** 8 indoor classes (person, chair, box, cone, pallet, shelf, hospital\_bed, cart), trained for 15 epochs at 640 px on script-drawn images.

- **v2.4:** 10 classes (`person`, `cart`, `forklift`, `pallet`, `box`, `obstacle`, `door`, `charging_station`, `hospital_bed`, `directional_sign`), trained for 25 epochs. A later audit found cross-split data leakage in its dataset (Section 5.1.1).
- **v2.5 (current):** the same 10 classes, trained for 30 epochs at batch size 16 and image size 640 on a rebuilt leak-free dataset with negative samples. YOLOv8n and YOLOv8s were compared and YOLOv8n was selected.

The v2.5 node (`object_detection_node.py`) reads its settings from `config/perception_v25.yaml`. It runs the model at  $640 \times 640$  on `cuda:0`, processes every second camera frame (`frame_skip = 2`), and uses an NMS IoU threshold of 0.45. Detections are filtered by a class-specific confidence threshold (default 0.35): 0.45 for `person`, `cart`, `obstacle` and `charging_station`; 0.50 for `forklift` and `door`; 0.40 for `pallet`, `box` and `directional_sign`; and 0.75 for `hospital_bed`, which was raised to suppress false detections on bare walls. A geometric filter then rejects boxes smaller than  $400 \text{ px}^2$ , larger than 60% of the frame area, or wider than 92% of its width, which removes full-frame background planes. At startup the node checks that the model’s class order matches the configuration.

### 4.2.2 Directional Sign Perception (Objective 6)

Directional signs are the `directional_sign` class of the detector. Each detected sign box is parsed into a destination and a direction and then passes two stages before it is published on `/vision/signs`:

1. a **temporal confirmation gate**: a sign must be observed in at least 3 frames with no more than 1.0 s between them, so that single-frame false detections never reach the Decision Engine; and
2. **landmark mapping**: the robot’s expected bearing to each sign in the semantic map (from the AMCL pose) is compared with the camera ray of the detection, and the sign is accepted if the bearing error is at most 0.40 rad and its range is between 0.5 and 6.0 m.

An empty sign array is published when no sign is confirmed, so that downstream nodes know when a sign has left the view. The original sign reader (`sign_detection_node`) instead matched candidate regions against 15 known sign textures by normalized cross-correlation, and this is the version exercised by scenario 1 of the test suite (Chapter 5). The v2.5 sign test set uses all 33 sign textures of the project; 16 signs, using 12 of these textures, are placed in the simulated facility.

### 4.2.3 LiDAR-Camera Sensor Fusion (Objective 4)

A monocular camera locates objects in the image plane but carries no depth; a 2D LiDAR measures range precisely but carries no semantic label. The `lidar_camera_fusion_node` takes the 20 Hz `/scan` and the `/vision/detections` and reconciles them in three steps. First, laser points are grouped into clusters by Euclidean distance (tolerance 0.25 m, at least 3 points), using a scan that is less than 100 ms older than the detection. Second, each YOLO bounding box is converted into an angular sector in the camera frame, linearly in the image column,

$$\theta_c = \left( \frac{1}{2} - \frac{u_c}{W} \right) \alpha, \quad \Delta\theta = \frac{u_{\max} - u_{\min}}{W} \alpha, \quad \theta_{\min, \max} = \theta_c \mp \frac{\Delta\theta}{2} \quad (4.1)$$

where  $u_{\min}$  and  $u_{\max}$  are the box's left and right pixel columns,  $u_c$  is their mean,  $W = 640$  px is the image width and  $\alpha = 1.15$  rad is the camera's horizontal field of view. Because the camera and the LiDAR are mounted at different points, the edges of the sector are projected into the LiDAR frame at five sample depths from 0.4 to 5.0 m and widened by 0.03 rad. The detection node, which reports the bearing of each object, uses the equivalent pinhole form with the focal length  $f_x = (W/2)/\tan(\alpha/2) \approx 494$  px. Third, the cluster on the camera ray is associated with the detection, which gives it a class label and a position in the `base_link` frame (the camera is mounted 0.22 m forward and 0.35 m above the robot origin). The result is published as `/vision/semantic_obstacles` for the Decision Engine and the costmaps. Each class has a safety radius, from 0.45 m for a box up to 1.10 m for a forklift, and the classes `person`, `cart` and `forklift` are treated as dynamic.

### 4.2.4 Object Tracking and Time-to-Collision (Objective 5)

Fused obstacles are tracked between frames by centroid distance: a new observation is matched to an existing track if it lies within 0.65 m, a track is dropped after 1.2 s without a match, and position and velocity are smoothed with an exponential moving average (weights 0.65 and 0.35). The time to collision of a tracked obstacle is estimated from its radial closing rate,  $\text{TTC} = r/(-\dot{r})$ , where  $r$  is its range and  $\dot{r}$  the rate of change of that range, and published on `/vision/ttc`. The Decision Engine's safety-yield callback halts the robot for 1.8 s whenever the estimate falls below 1.8 s. In v2.5 the closing rate is measured in the `base_link` frame, so a static object ahead of the moving robot appears to approach; V2.6 corrects the rate for the robot's own speed and yields only for dynamic obstacles (Section 5.1.15, Eq. 5.1).

### 4.2.5 Topological Memory and Decision Engine (Objectives 7, 8 and 9)

The facility’s topology is stored in a SQLite database (`navigation_memory.db`) that survives robot restarts. It uses write-ahead logging and has tables for nodes, edges, route segments, journeys, traversals, obstacles, recovery events and sign detections. The current facility graph has 53 nodes: 14 destinations, 8 junctions, 4 dead ends, 26 waypoints and the start node.

`topological_graph.py` finds the three lowest-cost routes between the start and the goal. The cost of an edge combines six terms and two penalties:

$$c(e) = w_d d + w_o \rho + w_h f + w_c \kappa + w_t n + w_n p + P_{\text{dead end}} + P_{\text{blocked}} \quad (4.2)$$

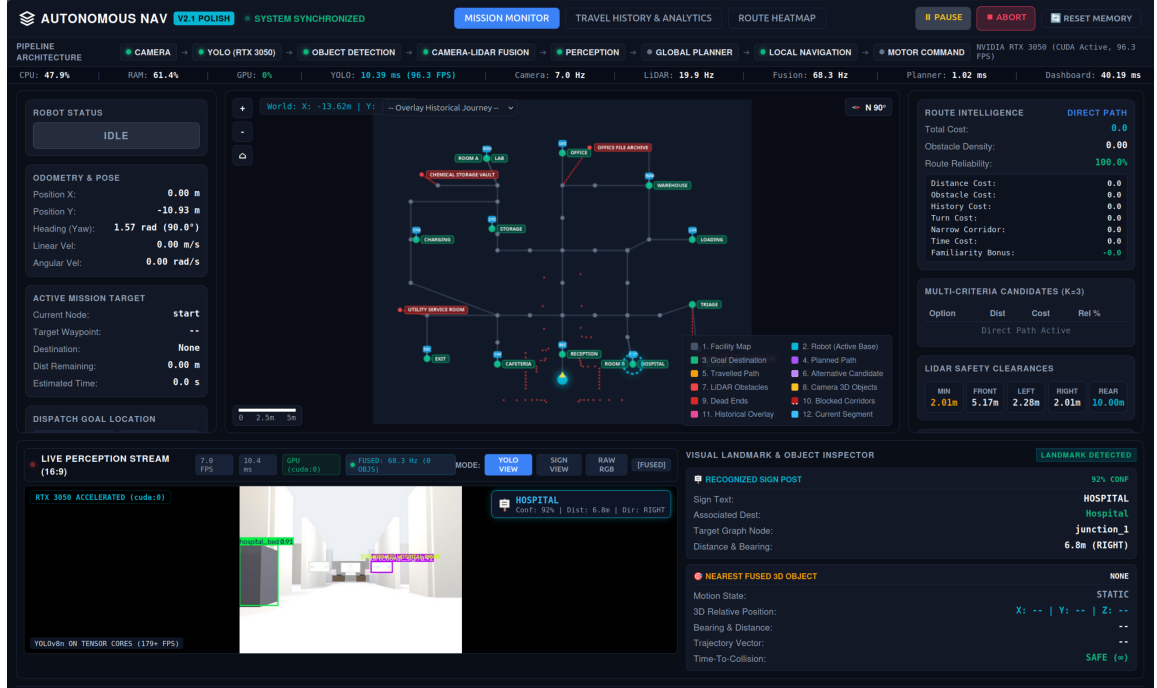
where  $d$  is the edge length,  $\rho$  the obstacle density,  $f$  a penalty for past failures,  $\kappa$  the congestion,  $n$  the number of turns and  $p$  a narrow-passage penalty. The weights are  $w_d = 1.0$ ,  $w_o = 8.0$ ,  $w_h = 5.0$ ,  $w_c = 3.0$ ,  $w_t = 2.0$  and  $w_n = 4.0$ ; the penalties are  $P_{\text{dead end}} = 10^5$  for an edge into a dead end and  $P_{\text{blocked}} = 10^6$  for an edge currently marked blocked.

The `decision_engine_node` turns this graph into motion. It subscribes to `/navigation/goal_label` (a room name such as `HOSPITAL`), selects the lowest-cost route, and sends the intermediate waypoints one after another to Nav2 as `/navigate_to_pose` goals, moving on to the next when the robot is within 0.60 m (in V2.6, within 0.75 m of an intermediate waypoint and 0.50 m of the final goal, without stopping at each waypoint). A confirmed sign that reads “closed” or points to a blocked path multiplies the weights of the incoming edges by 10, and a confirmed sign that matches the route is recorded in the memory. When fusion reports an obstacle in the forward corridor, the Decision Engine marks the edge blocked, cancels the active Nav2 goal, and replans.

### 4.2.6 Monitoring Dashboard (Objective 11)

A web dashboard (`dashboard_backend.py` with `web/index.html`, served at `127.0.0.1:5050`) shows the robot’s state in real time. It displays the camera view with the YOLO overlay, the detected objects, a visual-landmark panel for the current sign (text, direction, confidence, distance, associated destination and target node), the topological map with the destination highlighted, time-to-collision values and GPU telemetry. Sign distance is computed from the AMCL pose and the sign’s position in the semantic map, and a sign that has not been updated for 2.0 s is

removed from the display. Figure 4.3 shows the Mission Monitor view.



**Fig. 4.3.** The monitoring dashboard (Mission Monitor view) with the robot idle at the start node. Top: pipeline status and telemetry bar. Centre: the topological map with destinations and dead ends. Right: route-cost breakdown, multi-criteria route candidates and LiDAR clearances. Bottom: the annotated camera view with a `hospital_bed` and a `directional_sign` detection, and the visual-landmark panel showing a confirmed HOSPITAL sign (92% confidence, 6.8 m, direction right, target node `junction_1`) with the nearest fused object and time to collision.

## 4.3 Design

The design follows the layered split shown in Fig. 4.1: Nav2’s NavFn planner and Regulated Pure Pursuit controller handle metric planning and trajectory execution, while the Decision Engine handles everything semantic. The penalty-based (rather than edge-deletion) treatment of dead ends and blockages keeps the graph connected, so a route always exists. Table 4.2 gives the main Nav2 settings used in the v2.5 benchmark and, where V2.6 changed them, the V2.6 value.

**Table 4.2** Main Nav2 configuration (`nav2_params.yaml`; v2.5 value  $\rightarrow$  V2.6 value where changed)

| Component                                    | Setting                                         | Value                                                       |
|----------------------------------------------|-------------------------------------------------|-------------------------------------------------------------|
| Global planner<br>(NavFn)                    | A* search; goal tolerance;<br>unknown cells     | enabled; 1.5 m; allowed                                     |
| Local controller<br>(Regulated Pure Pursuit) | Desired speed; look-ahead<br>distance           | 0.50 m/s; 0.60 m (0.30–0.90 m)                              |
|                                              | Cost-regulated speed<br>scaling; minimum speed  | enabled; 0.10 $\rightarrow$ 0.15 m/s                        |
|                                              | Approach scaling distance;<br>final speed       | 0.60 $\rightarrow$ 0.45 m; 0.05 $\rightarrow$ 0.08 m/s      |
|                                              | Collision look-ahead before<br>fast stop        | 2.0 $\rightarrow$ 1.2 s                                     |
|                                              | Rotate-to-heading speed                         | 1.00 rad/s                                                  |
| Local costmap                                | Rolling window; update<br>rate                  | 6 $\times$ 6 m; 5 Hz                                        |
| Global costmap                               | Update rate                                     | 1 Hz                                                        |
| Both costmaps                                | Inflation radius; cost<br>scaling; robot radius | 0.38 $\rightarrow$ 0.30 m; 4.0 $\rightarrow$ 4.5;<br>0.22 m |
| Progress checker                             | Required movement /<br>allowance                | 0.50 m in 10 s                                              |
| Goal checker                                 | Position / heading<br>tolerance                 | 0.35 m / 0.50 rad                                           |

## 4.4 Materials and Methods

Table 4.3 summarizes the robot, its sensors, and the software and compute environment.

**Table 4.3** Robot, sensors and environment

| Component                      | Specification                                                                                                      |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Platform                       | Differential-drive ground robot: two drive wheels (radius 0.06 m, track 0.35 m) and two passive casters            |
| Chassis                        | $0.45 \times 0.30 \times 0.15$ m, 6.63 kg (derived); Nav2 footprint is a circle of 0.22 m radius                   |
| Drive limits                   | Linear $\pm 0.8$ m/s, angular $\pm 2.0$ rad/s                                                                      |
| Camera                         | RGB $640 \times 480$ at 30 Hz, mounted 0.22 m forward and 0.35 m above the base, horizontal field of view 1.15 rad |
| LiDAR                          | Planar, $360^\circ$ , 720 samples ( $0.5^\circ$ ) at 20 Hz, range 0.12–12.0 m, mounted 0.175 m above the base      |
| Compute                        | NVIDIA RTX 3050 Laptop GPU (4 GB)                                                                                  |
| Middleware and simulator       | ROS 2 Lyrical, Nav2, Gazebo Sim 10.5                                                                               |
| Operating system and container | Ubuntu 26.04 LTS; <code>nvidia/cuda:13.3.1-runtime-ubuntu26.04</code> base image (CUDA 13.0)                       |
| Detector and memory            | Ultralytics YOLOv8n; SQLite                                                                                        |

#### 4.4.1 Simulated Facility

All experiments run in a simulated multi-room facility (`realistic_facility_world.sdf`) of  $32 \times 26$  m, with wards, a warehouse, a storage depot, a lab, an office, a cafeteria and a reception area, on a  $36 \times 30$  m ground plane. It is mapped as a 0.05 m occupancy grid ( $700 \times 600$  pixels). According to the project’s object inventory (Annexure II), the world has 93 models: 4 dynamic agents, 29 pieces of static furniture and equipment, 16 wall-mounted directional signs, and 43 structural walls and pillars. Lighting is one directional sun and 10 ceiling lights. All 10 detector classes are represented in the world, and the eleven named destinations are listed in Annexure II.

Corridors are narrow: 1.8 m in the east and west corridors, 1.4 m on the west bypass and the utility corridor, and 3.6 m in the central spine. The hospital ward doorway is 1.5 m wide, and the robot’s start pose in the ward leaves only 0.5 m between a bed and the doorway.

The four dynamic agents patrol fixed paths and are the main source of time-to-collision yields and replanning (Table 4.4). The hospital trolley crosses the hospital ward doorway and blocks it periodically.

**Table 4.4** Dynamic agents in the simulated facility

| Agent (class)                          | Patrol path                   | Speed (m/s) | Safety radius (m) |
|----------------------------------------|-------------------------------|-------------|-------------------|
| Warehouse cart ( <code>cart</code> )   | $X = 8.0, Y \in [3.5, 5.25]$  | 0.35        | 0.75              |
| Hospital trolley ( <code>cart</code> ) | $Y = -6.5, X \in [5.0, 7.0]$  | 0.25        | 0.75              |
| Forklift ( <code>forklift</code> )     | $X = 11.5, Y \in [4.5, 5.82]$ | 0.22        | 1.10              |
| Walking person ( <code>person</code> ) | $X = -6.0, Y \approx -6.5$    | 0.30        | 0.90              |

## 4.5 Data Acquisition

All training data is synthetic. For v1, `scripts/generate_synthetic_dataset.py` draws each class as flat OpenCV shapes on randomized two-tone backgrounds and writes exact YOLO labels (200 images, 408 objects). The v2.4 dataset had 1,200 images (840/240/120 split).

The v2.5 dataset is generated by `scripts/generate_v25_dataset.py`. It holds 2,000 images (1,400 train, 400 validation, 200 test) at  $640 \times 480$  in YOLO format. Of these, 15% are empty backgrounds (corridor views, wall junctions, floor textures and shadow lines) with no annotations. It draws the actual Gazebo facility assets, including the orange transport cart, the cyan hospital trolley, the yellow forklift, the blue person cylinder, wooden pallets, box stacks, the charging dock and the hospital beds, plus all 33 sign textures rendered with perspective at near (0.8–2.0 m), medium (2.0–4.2 m) and far (4.5–7.5 m) distances and at frontal (0–10°), slightly oblique and highly oblique (25–45°) angles. Non-overlapping random seeds are used for each split, and a perceptual-hash (`dhash`) check confirmed that no image is duplicated across splits.

## 4.6 Implementation

The reference environment is containerized in Docker so that the whole stack (ROS 2 Lyrical, Gazebo Sim 10.5 and the PyTorch/YOLO perception) can be reproduced on any host. The container is built from the official `nvidia/cuda:13.3.1-runtime-ubuntu26.04` base image, layers the `ros-lyrical-*` apt packages published for Ubuntu 26.04, and is run with `-gpus all`, so that inference gets native GPU acceleration through the NVIDIA Container Toolkit, confirmed with `nvidia-smi` inside the container. The perception node, the fusion node, the Decision Engine, Nav2, Gazebo and the dashboard are started together by the project’s launch scripts, which integrates the complete stack (Objective 10).

Paths resolve from a `CV_NAV_ROOT` variable that `start_project.sh` exports, falling back to each file's own location. Bringing the workspace up from a clean clone surfaced six defects, all fixed; they are documented in Annexure I.

# Chapter 5

## Results and Discussion

The results fall into three tiers of evidence: some were re-run and verified on a second machine, some are recorded in the repository with the generating script and output file present, and some are reported only, meaning they are stated in the project documentation with no log or data file behind them. The tier of each result is stated where it is presented. The detector results in Sections 5.1.2 to 5.1.6 are from the project’s v2.5 release (tag `v2.5.0`), measured on an NVIDIA RTX 3050 Laptop GPU. The V2.6 fixes and experiments in Section 5.1.15 are from the V2.6 release (tag `v2.6.0`, commit `25bab3f` on the `v2.6-dev` branch).

### 5.1 Presentation of Results

#### 5.1.1 Detector Progression and the Data-Leakage Finding

Three generations of the detector were trained. The first (v1) used 200 script-drawn images and scored 96.2% mAP@0.50 in-distribution. The second (v2.4) used 1,200 images and originally reported 0.910 mAP@0.50 on its test split. The v2.5 work audited the v2.4 dataset with perceptual hashing (`dhash`) and found 83 near-duplicate image pairs, of which 35 crossed the train/validation/test boundaries. The 0.910 figure was therefore inflated by leakage. Evaluated on the leak-free v2.5 validation split, the v2.4 model scores only 0.634 mAP@0.50 (Table 5.2). The v2.4 audit also found that the dataset contained no empty-background images and that its cart and forklift drawings did not match the Gazebo assets (8.7% cart recall and 48.3% forklift recall on the facility assets). The v1 and v2.4 headline numbers are not used as evidence in this report.

### 5.1.2 YOLOv8n v2.5: Dataset and Training

The v2.5 dataset holds 2,000 images (1,400 train, 400 validation, 200 test) across the same 10 classes, of which 15% (300 images) are pure negative backgrounds with no annotations. Scene seeds are disjoint between splits, and a perceptual-hash check found 0 cross-split duplicates. It renders the actual Gazebo assets (transport cart, hospital trolley, forklift, person cylinder, pallets, box stacks, charging dock, hospital beds) and all 33 physical sign textures, with near, medium and far distances and frontal and oblique angles. Two models were trained for 30 epochs at batch size 16 and image size 640 on the RTX 3050: YOLOv8n (training time 480 s) and YOLOv8s. YOLOv8n was selected as the production model (`models/yolov8n_v25.pt`): its validation mAP@0.50 (0.8994) is equal to YOLOv8s (0.8970), but it has 3.01M parameters against 11.13M and runs 2.4 times faster (4.06 ms against 9.82 ms per image) with a smaller memory footprint.

### 5.1.3 YOLOv8n v2.5: Detection Accuracy

Overall results are in Table 5.1 and per-class results in Table 5.2. The source is `yolov8n_v25_metrics.json`.

**Table 5.1** Overall performance of YOLOv8n v2.5 on the leak-free validation and test splits

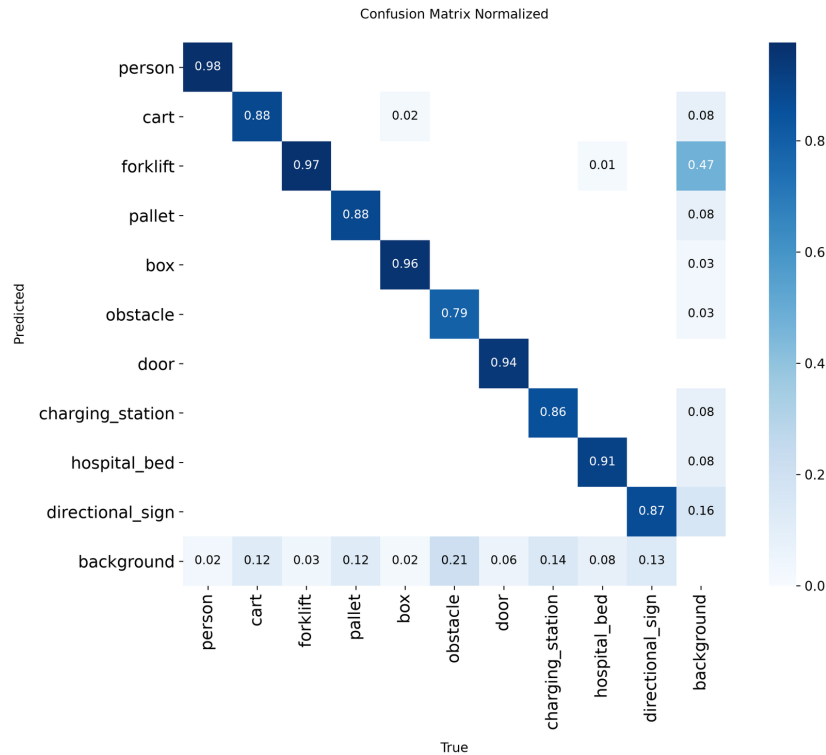
| Split                   | Precision | Recall | mAP@0.50 | mAP@0.50–0.95 | Inference |
|-------------------------|-----------|--------|----------|---------------|-----------|
| Validation (400 images) | 0.9823    | 0.8884 | 0.8994   | 0.8574        | 4.06 ms   |
| Test (200 images)       | 0.9765    | 0.8911 | 0.8991   | 0.8627        | 4.43 ms   |

**Table 5.2** Per-class results of YOLOv8n v2.5, with the v2.4 model on the same leak-free validation split for comparison

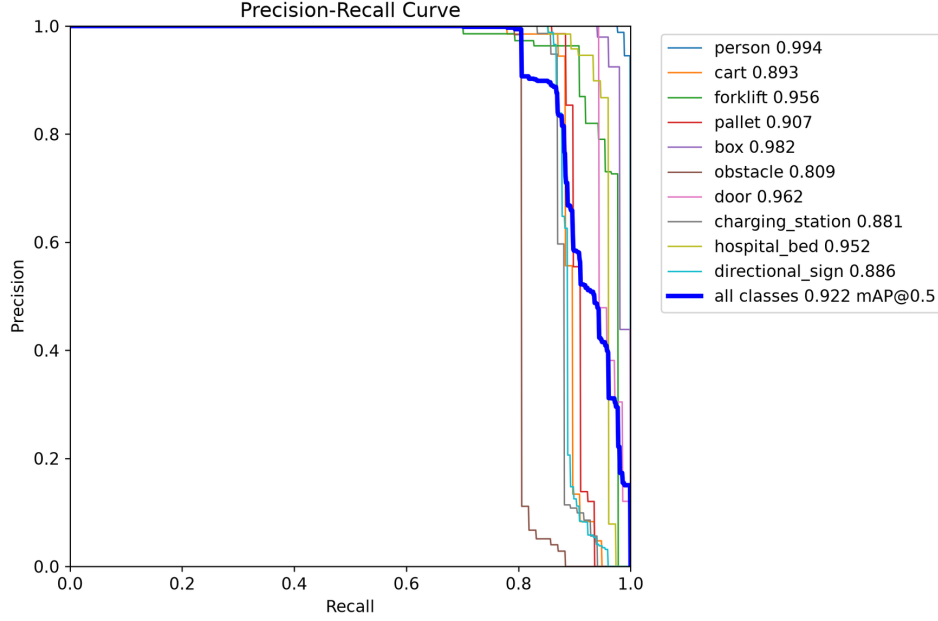
| Class                     | v2.4 AP@0.50<br>(val) | v2.5 AP@0.50<br>(val) | v2.5 Precision<br>(val) | v2.5 Recall<br>(val) | v2.5 AP@0.50 (test) |
|---------------------------|-----------------------|-----------------------|-------------------------|----------------------|---------------------|
| Person                    | 0.927                 | 0.975                 | 1.000                   | 0.971                | 0.865               |
| Cart                      | 0.052                 | 0.883                 | 0.955                   | 0.870                | 0.875               |
| Forklift                  | 0.456                 | 0.933                 | 0.962                   | 0.882                | 0.840               |
| Pallet                    | 0.433                 | 0.885                 | 0.969                   | 0.885                | 0.905               |
| Box                       | 0.838                 | 0.955                 | 0.991                   | 0.940                | 0.995               |
| Obstacle                  | 0.805                 | 0.795                 | 0.997                   | 0.792                | 0.845               |
| Door                      | 0.912                 | 0.945                 | 1.000                   | 0.944                | 0.955               |
| Charging Station          | 0.462                 | 0.855                 | 0.986                   | 0.853                | 0.913               |
| Hospital Bed              | 0.603                 | 0.904                 | 0.975                   | 0.893                | 0.963               |
| Directional Sign          | 0.852                 | 0.865                 | 0.988                   | 0.853                | 0.835               |
| <b>Overall (mAP@0.50)</b> | <b>0.634</b>          | <b>0.899</b>          | <b>0.982</b>            | <b>0.888</b>         | <b>0.899</b>        |

The v2.4 column is read directly from `v24_on_v25_benchmark.json`; the model itself was not re-run for this report. Cart, forklift, pallet, and charging-station detection improve the most, which matches the audit’s finding that v2.4 had never seen the actual Gazebo asset shapes. **obstacle** is the one class that did not improve (0.805 to 0.795, recall 0.792). The gap between validation and test AP for **person** (0.975 against 0.865) and **directional\_sign** (0.865 against 0.835) shows that per-class estimates on 30–100 test instances are noisy.

The normalized confusion matrix and the precision–recall curve of the v2.5 model were published with the V2.6 release (`report_evidence/v26_final/objective02_yolo`) and are shown in Figs. 5.1 and 5.2. They were produced by Ultralytics at the end of training; the model was not re-run for this report (recorded in the repo). The diagonal of the confusion matrix agrees with the per-class recall in Table 5.2, from 0.98 for **person** down to 0.79 for **obstacle**. There is almost no confusion between object classes; the errors are with the background. The detector misses 21% of **obstacle** instances and 12–14% of **cart**, **pallet** and **charging\_station** instances, and the background regions it wrongly detects are most often labelled **forklift** (47%) or **directional\_sign** (16%).



**Fig. 5.1.** Normalized confusion matrix of YOLOv8n v2.5 on the validation split (columns: true class; rows: predicted class; the last row and column are the background).



**Fig. 5.2.** Precision–recall curves of YOLOv8n v2.5 on the validation split, per class and for all classes (mAP@0.50 = 0.922 as computed at the end of training).

The curve gives an all-class mAP@0.50 of 0.922, against 0.899 in the separate evaluation of Table 5.1, and its background column does not match the background false-positive counts in the metrics file, where `directional_sign` (26) and `obstacle` (16) are the most frequent. The two were produced with different evaluation settings, which are not documented. This report uses 0.899 as the headline figure.

#### 5.1.4 Robustness Across Distance, Angle, Lighting, Blur and Occlusion

Nine 60-sample stress subsets were evaluated (Table 5.3).

**Table 5.3** mAP@0.50 on nine stress subsets (60 samples each)

| Condition                     | v2.4  | v2.5 (YOLOv8n) |
|-------------------------------|-------|----------------|
| Distance: near (0.8–2.0 m)    | 0.688 | 0.902          |
| Distance: medium (2.2–4.2 m)  | 0.673 | 0.895          |
| Distance: far (4.5–7.5 m)     | 0.660 | 0.879          |
| Orientation: frontal          | 0.618 | 0.911          |
| Orientation: oblique (20–45°) | 0.670 | 0.887          |
| Lighting: bright (1.30×)      | 0.625 | 0.867          |
| Lighting: dim (0.65×)         | 0.608 | 0.887          |
| Motion blur (0.5 m/s)         | 0.654 | 0.887          |
| Partial occlusion             | 0.608 | 0.859          |

### 5.1.5 Controlled Perception Tests

`scripts/test_controlled_perception_cases.py` runs eight targeted tests on ground-truth and synthetic images with the deployed post-processing: per-class confidence thresholds (for example 0.75 for `hospital_bed`) and a geometric filter that rejects boxes covering more than 60% of the frame area or more than 92% of its width. Results are in Table 5.4.

**Table 5.4** Controlled perception tests (`controlled_perception_tests_v25.json`)

| Test | Description                     | Samples | Result              | Script status |
|------|---------------------------------|---------|---------------------|---------------|
| A    | Empty hallway                   | 30      | 0 false positives   | Pass          |
| B    | Empty wall (diffuse highlights) | 20      | 0 false positives   | Pass          |
| C    | Floor, corner and shadow        | 15      | 0 false positives   | Pass          |
| D    | Hospital bed                    | 30      | 27 detected (90.0%) | Pass          |
| E    | Cart / trolley                  | 30      | 22 detected (73.3%) | <b>Fail</b>   |
| F    | Forklift                        | 30      | 23 detected (76.7%) | <b>Fail</b>   |
| G    | Person                          | 30      | 30 detected (100%)  | Pass          |
| H    | Directional signs, overall      | 100     | 58 detected (58.0%) | <b>Fail</b>   |
|      | near sign (0.8–2.0 m)           | 20      | 7 detected (35.0%)  |               |
|      | medium                          | 20      | 13 detected (65.0%) |               |
|      | far                             | 20      | 14 detected (70.0%) |               |
|      | frontal                         | 20      | 13 detected (65.0%) |               |
|      | oblique                         | 20      | 11 detected (55.0%) |               |

The wall false positive that motivated the tests (a `hospital_bed` at about 0.68

confidence on a bare wall) is removed by the threshold and the geometric filter without lowering the recall on true beds. However, the tests also show that the deployed detector misses about a quarter of carts and forklifts and 42% of directional signs in these conditions, and that signs are missed most often when they are near (35% recall). The project’s validation report marks tests E, F and H as “PASS”, while the results file itself marks them “FAIL”; this report follows the results file.

### 5.1.6 Live ROS 2 Pipeline Checks

A ROS 2 test that published 119 frames over `/camera/image_raw` at 20 Hz received detections for 118 of them, loaded the model on `cuda:0`, and confirmed that the class order matches the configuration. The end-to-end node latency was 7.83 ms including tensor preparation and non-maximum suppression. The same test found 0 false positives on 60 empty-hallway images and recognized all 33 sign textures (100%, mean confidence 0.988), and 16 signs passed the temporal confirmation gate (a sign must be seen in at least 3 frames with no more than 1.0 s between them). Telemetry from the running Gazebo simulation showed 14.8 ms inference, 33.8 Hz effective perception rate, 12% GPU utilization, and 173 MB of GPU memory. The 100% result here and the 58% sign recall in Table 5.4 come from different tests and are not reconciled in the project documentation.

### 5.1.7 Directional Sign Perception

Scenario 1 of the older test suite pastes each of 15 sign textures at 50% scale onto a flat gray  $640 \times 480$  image and requires the node to decode the correct destination and arrow. All 15 pass, with mean confidence 0.99 and under 4 ms per frame. This result is recorded in the repository but was not re-run on the second machine, since scenario 1 imports `rclpy` and must run inside the ROS container. It checks the decoding logic on constructed inputs, not recognition at distance and angle (see Table 5.4 for that).

### 5.1.8 LiDAR-Camera Fusion Accuracy

The figures in Table 5.5 come from `scripts/test_lidar_fusion.py`, which is algorithmic validation rather than Gazebo data: it places a person-sized box straight ahead at four distances and generates a matching LiDAR scan with 1.5 cm Gaussian noise added to the true range.

**Table 5.5** Fusion range estimate against ground truth (synthetic scan, 1.5 cm noise)

| Ground truth (m) | Estimated (m) | Absolute error (m) | Error (%) |
|------------------|---------------|--------------------|-----------|
| 1.00             | 0.9909        | 0.0091             | 0.91      |
| 1.50             | 1.4889        | 0.0111             | 0.74      |
| 2.00             | 1.9875        | 0.0125             | 0.62      |
| 2.50             | 2.4922        | 0.0078             | 0.31      |

### 5.1.9 Replanning Performance

The block-and-replan step was timed on the 38-node graph that was in use when the test was run (the current graph has 53 nodes), over 2,000 runs per case (Table 5.6). This is the graph search alone, on a cloud x86 CPU rather than the project laptop; detecting the obstacle and cancelling the Nav2 goal add their own delay on top.

**Table 5.6** Block-and-replan timing on the 38-node graph (2,000 runs per case)

| Blocked edge               | Median   | 99th percentile | Worst    |
|----------------------------|----------|-----------------|----------|
| wh_junction_6 -> warehouse | 0.032 ms | 0.110 ms        | 0.246 ms |
| junction_7 -> junction_8   | 0.063 ms | 0.229 ms        | 0.936 ms |

In scenario 2, blocking `junction_7 -> junction_8` on the way to the lab sends the robot around the West Bypass (via `lab_corridor_entry`, `junction_4`, `central_spine_1`, `junction_1`, `corridor_west_1`, `junction_3`, `west_bypass_south`, `west_bypass_north`, and `junction_8`), and unblocking the edge restores the direct route.

#### 5.1.10 Four-Scenario Test Suite

The four-scenario suite (`scripts/run_all_experiments.py`) was re-run from a different checkout path after the portability fixes in Annexure I. Scenarios 2–4 exercise the real planning code and need no ROS runtime.

**Table 5.7** Four-scenario test suite results

| Scenario              | What it checks                                                                    | Result                           | Evidence                          |
|-----------------------|-----------------------------------------------------------------------------------|----------------------------------|-----------------------------------|
| 1. Sign perception    | Each of 15 textures, pasted on a gray image, decodes to the right label and arrow | 15/15                            | Recorded in repo; not re-run here |
| 2. Dynamic replanning | Blocking an edge forces a detour; unblocking restores the route                   | Pass (warehouse and West Bypass) | Re-run and verified               |
| 3. Dead-end avoidance | Routes to all 7 destinations avoid the 4 dead-end nodes                           | 0 dead-end entries               | Re-run and verified               |
| 4. Memory persistence | Nodes, edges, signs, and blockages survive a reload; reset keeps topology         | Pass                             | Re-run and verified               |

### 5.1.11 Ten-Mission Navigation Benchmark in Gazebo

The official benchmark `scripts/run_v24_10_missions.py` runs ten start-to-goal missions through the full system in Gazebo with the v2.5 detector. Each mission has a 180 s limit (mission 10 is given 220 s by line 289 of the script, which was also used for the final v2.4 run; the reason is not documented). Results are in Table 5.8.

**Table 5.8** Ten-mission benchmark with the v2.5 detector (from the project’s V2.5 validation documentation; totals as recorded in `v24_mission_validation_results.json`)

| #            | Start     | Goal     | Time (s) | Distance (m) | Min. clearance (m) | TTC yields | Replans    |
|--------------|-----------|----------|----------|--------------|--------------------|------------|------------|
| 1            | Room A    | Storage  | 180.4    | 2.99         | 0.28               | 12         | 52         |
| 2            | Room A    | Hospital | 180.1    | 23.31        | 0.56               | 0          | 41         |
| 3            | Loading   | Charging | 180.0    | 39.02        | 0.19               | 3          | 21         |
| 4            | Warehouse | Exit     | 180.1    | 9.89         | 0.57               | 8          | 36         |
| 5            | Storage   | Office   | 180.0    | 45.47        | 0.34               | 20         | 13         |
| 6            | Hospital  | Storage  | 180.4    | 6.72         | 0.29               | 17         | 50         |
| 7            | Charging  | Exit     | 180.1    | 35.60        | 0.48               | 7          | 14         |
| 8            | Warehouse | Hospital | 180.2    | 34.54        | 0.47               | 7          | 22         |
| 9            | Room B    | Storage  | 180.2    | 2.81         | 0.30               | 7          | 43         |
| 10           | Reception | Room A   | 220.1    | 30.49        | 0.12               | 14         | 35         |
| <b>Total</b> |           |          |          | <b>230.8</b> |                    | <b>95</b>  | <b>327</b> |

All ten missions ended in a timeout: the mission success rate is **0 of 10**. There were 0 collisions, 0 stuck events and 0 manual interventions, and the robot confirmed 29 sign observations across the run. Missions 1, 6 and 9 start in the hospital ward

and make almost no progress (2.8–6.7 m in 180 s) with 43–52 replans each. In the other missions the robot travels 23–45 m, but at an effective speed far below the configured 0.50 m/s it does not reach the goal within the time limit. The causes found in the project’s audit are listed in Table 5.17.

The same benchmark had been run twice with the v2.4 detector (Table 5.9). The final v2.4 run (after the doorway widening of Step G, commit `c732938`) used the same world, script and time limits as v2.5, so it is the like-for-like comparison. The earlier Step F baseline ran in the older world, before the doorways were widened, with a 160 s limit for every mission, and is listed for completeness only. Against the final v2.4 run, v2.5 cut TTC yields from 300 to 95 and replans from 353 to 327, and travelled further (230.84 m against 146.23 m) at a higher effective speed (0.125 against 0.085 m/s), but mission success *fell* from 2 of 10 to 0 of 10.

**Table 5.9** Ten-mission benchmark: the two v2.4 runs against v2.5 (all values read directly from the results files)

| Metric                                           | v2.4 Step F baseline                     | v2.4 final (Step G)                         | v2.5                                        |
|--------------------------------------------------|------------------------------------------|---------------------------------------------|---------------------------------------------|
| Results file                                     | <code>v24_step_f_baseline_results</code> | <code>v24_mission_validation_results</code> | <code>v24_mission_validation_results</code> |
| Commit                                           | <code>c732938</code>                     | <code>c732938</code>                        | <code>0ef2f8b</code>                        |
| World                                            | before doorway widening                  | after doorway widening                      | after doorway widening                      |
| Time limit                                       | 160 s                                    | 180 s (M10: 220 s)                          | 180 s (M10: 220 s)                          |
| Missions completed                               | 3 / 10                                   | 2 / 10                                      | 0 / 10                                      |
| Missions timed out                               | 7                                        | 8                                           | 10                                          |
| Total distance travelled                         | 273.07 m                                 | 146.23 m                                    | 230.84 m                                    |
| Total run duration                               | 1332.6 s                                 | 1715.6 s                                    | 1841.6 s                                    |
| Effective velocity                               | 0.205 m/s                                | 0.085 m/s                                   | 0.125 m/s                                   |
| Replanning events                                | 111                                      | 353                                         | 327                                         |
| TTC safety yields                                | 30                                       | 300                                         | 95                                          |
| Collisions / manual interventions / stuck events | 0 / 0 / 0                                | 0 / 0 / 0                                   | 0 / 0 / 0                                   |
| Minimum LiDAR clearance                          | 0.13 m                                   | 0.19 m                                      | 0.12 m                                      |
| Sign detections logged                           | 0                                        | 34                                          | 29                                          |

The v2.5 run was written to the file name used by the v2.4 script, so at commit `0ef2f8b` `v24_mission_validation_results.json` holds the v2.5 results (its notes still read “V2.4”); the final v2.4 results were read from the same file at commit `c732938`.

### 5.1.12 Ablation Study

`scripts/run_ablation_study.py` compares three obstacle-handling configurations across 5 hand-written scenarios. It is a standalone 2D Python simulation of the robot’s path, not a Gazebo or Nav2 run (Table 5.10).

**Table 5.10** Ablation of obstacle-handling modalities (2D simulation, 5 scenarios)

| Modality          | Success | Collisions | Replans | Avg. path | Avg. time | Min. clearance |
|-------------------|---------|------------|---------|-----------|-----------|----------------|
| LiDAR only        | 100.0%  | 0          | 6       | 14.67 m   | 31.74 s   | 0.31 m         |
| Camera/CV only    | 100.0%  | 0          | 8       | 14.89 m   | 32.99 s   | 0.66 m         |
| LiDAR + CV fusion | 100.0%  | 0          | 7       | 14.89 m   | 32.59 s   | 0.66 m         |

### 5.1.13 Full-Facility Traversal (Reported Only)

The project README describes a full-facility traversal of 29.18 m in 73.77 s at up to 0.5 m/s, with 0.34 m minimum LiDAR clearance. The repository contains no log, rosbag, or CSV from that run, so these figures could not be checked and are not treated as evidence in this report. They are also hard to reconcile with the 0 of 10 mission result in Section 5.1.11.

### 5.1.14 Comparative Analysis

The detector comparison in Table 5.2 is between two models on the same leak-free split, so it is like for like: v2.5 improves overall mAP@0.50 from 0.634 to 0.899. Its speed is unchanged, as expected for the same YOLOv8n architecture: on the same benchmark v2.4 takes 3.94 ms (253.9 FPS) and v2.5 4.06 ms (246.2 FPS) per image, and both use about 175 MB of GPU memory in the running node. On the original, leaky v2.4 split the two models are reported to score 0.912 (v2.4) and 0.899 (v2.5) (reported only), so v2.5 is slightly lower there; this again shows that the leaky split flatters v2.4. The comparison with the earlier headline numbers (96.2% for v1, 91.0% for v2.4) is therefore not valid, because those were measured in-distribution or on a leaky split. No literature benchmark comparison has been made.

The mission comparison in Table 5.9 is the opposite: perception improved but navigation performance dropped. The final v2.4 run completed 2 of 10 missions, the v2.5 run none, so the better detector did not translate into better mission results. Section 5.2.3 explains why.

### 5.1.15 V2.6 Reliability Fixes

The V2.6 release implements the fixes that the audit of v2.5 called for (Table 5.17). All of them were confirmed in the code at tag `v2.6.0`:

- **Ego-motion-compensated TTC.** The fusion node now subscribes to `/odom` and corrects the radial rate of each tracked obstacle for the robot’s own forward speed  $v_{\text{robot}}$  (Eq. 5.1). An obstacle with a compensated speed below 0.08 m/s

is classed as static and gets no TTC; a yield now also requires a dynamic class or a true speed above 0.12 m/s, and a closing speed above 0.08 m/s inside the robot’s forward corridor.

- **Stuck timer suspended during yields.** The Decision Engine refreshes its progress timer while a TTC or dynamic-obstacle pause is active, so a yield no longer triggers a stuck recovery.
- **Session memory reset.** `reset_transient_blockages()` clears the temporary blocked-edge flags at the start of a mission and keeps the traversal statistics.
- **Nav2 tuning.** The inflation radius of both costmaps is reduced from 0.38 to 0.30 m (cost scaling 4.0 to 4.5); the minimum regulated speed is raised from 0.10 to 0.15 m/s; the approach scaling distance falls from 0.60 to 0.45 m; and the controller’s collision look-ahead falls from 2.0 to 1.2 s (Table 4.2).
- **Waypoint handoff.** An intermediate waypoint is accepted within 0.75 m (0.50 m for the final goal), and the next one is sent without first cancelling the active Nav2 goal.
- **Start poses.** The hospital and Room B start poses, which share one position in the ward, are moved to the centre aisle, (5.0, −9.0), facing the door, and the lab, Room A and loading start poses are turned by 180°.

$$\dot{r}_{\text{true}} = \dot{r}_{\text{apparent}} + v_{\text{robot}} \cos \theta \quad (5.1)$$

where  $\dot{r}_{\text{apparent}}$  is the rate of change of the obstacle’s range measured in the `base_link` frame and  $\theta$  its bearing. For a static obstacle straight ahead of a robot moving at  $v_{\text{robot}}$ ,  $\dot{r}_{\text{apparent}} = -v_{\text{robot}}$  and so  $\dot{r}_{\text{true}} = 0$ .

### 5.1.16 V2.6 Controlled Experiments

The V2.6 fixes were tested with short experiments in Gazebo, each with its own script and results file in `results/v26_phase2` (recorded in the repo).

**Runtime TTC tests.** `scripts/test_ttc_runtime.py` drives the robot past a static bed, into the path of a moving cart and past the patrolling person, and cruises along the central spine at 0.50 m/s to test the ego-motion compensation. The test was run twice (Table 5.11). All four cases pass in the first run. In the later run

recorded in `results/v26_final`, the ego-motion case fails with one false yield. In both runs the file records that the cart itself was not classified by the detector, so the yield in test B was triggered by a tracked LiDAR obstacle without its class.

**Table 5.11** Runtime TTC tests in Gazebo after the V2.6 fixes (two runs)

| Test                           | Run 1 ( <code>v26_phase2</code> )       | Run 2 ( <code>v26_final</code> )        |
|--------------------------------|-----------------------------------------|-----------------------------------------|
| A. Static hospital bed         | Pass: 0 false yields, no TTC            | Pass: 0 false yields, no TTC            |
| B. Moving cart                 | Pass: TTC 1.02 s, 0.47 s yield, resumed | Pass: TTC 0.97 s, 2.09 s yield, resumed |
| C. Patrolling person / trolley | Pass: tracked, minimum TTC 2.16 s       | Pass: tracked, minimum TTC 9.13 s       |
| D. Ego-motion at 0.50 m/s      | Pass: 0 false yields                    | <b>Fail: 1 false yield</b>              |

**Inflation sweep.** `scripts/experiment_inflation_clearance.py` ran four start-goal pairs for 55 s each at four inflation radii (Table 5.12). No run reached its goal in the 55 s window, and none had a collision. The reduction to 0.30 m helped the reception run most (20.20 m at 0.367 m/s, against 16.39 m at 0.297 m/s at 0.38 m), but at no radius did the robot leave the hospital ward, where it travelled at most 1.27 m.

**Table 5.12** Inflation-radius sweep: distance travelled in 55 s (m), with minimum clearance (m) and replans in brackets

| Start → goal       | 0.38 m          | 0.32 m          | 0.30 m          | 0.28 m          |
|--------------------|-----------------|-----------------|-----------------|-----------------|
| Hospital → Storage | 1.27 (0.28; 16) | 0.38 (0.38; 10) | 0.41 (0.39; 13) | 0.83 (0.38; 13) |
| Warehouse → Exit   | 10.49 (0.74; 8) | 7.07 (0.71; 8)  | 8.63 (0.72; 8)  | 0.00 (0.59; 14) |
| Storage → Office   | 11.55 (0.90; 3) | 15.58 (0.20; 5) | 9.62 (0.30; 11) | 14.25 (0.19; 5) |
| Reception → Room A | 16.39 (0.27; 4) | 15.75 (0.26; 1) | 20.20 (0.57; 2) | 20.09 (0.44; 2) |

**Hospital start pose.** `scripts/experiment_hospital_start.py` tested three start poses in the ward for 35 s each: the v2.5 pose facing away from the door, the same position facing the door, and the centre-aisle pose facing the door. All three ended trapped in the ward (0.86, 0.34 and 0.00 m travelled, 0 collisions). The centre-aisle pose was nevertheless adopted in `navigation_locations.yaml`.

**Controller speed.** On the straight central spine from reception towards the office, the tuned controller covered 16.32 m in 35.02 s at an average of 0.472 m/s (94.4% of the 0.50 m/s set speed), and spent 32.34 s above 0.40 m/s. This confirms that the speed limit seen in v2.5 comes from corridor costs and waypoint stops, not from the controller itself.

**Waypoint handoff.** Only the baseline run, before the change, is recorded: 19.24 m in 55.14 s through 7 waypoints, of which 12.25 s was spent stationary. No run with the new handoff is recorded, so its effect is not measured.

**Live replanning.** `scripts/experiment_dynamic_replanning.py` blocked the edge `corridor_east_1 -> junction_2` on a mission from reception to the warehouse. The route changed from the east bypass (22.84 m, cost 870.48) to the central spine and the office–warehouse bypass (23.60 m, cost 873.32), a detour of 0.76 m. The run had no replan cascade, no TTC yield and no collision, with a minimum clearance of 1.26 m. This is a single scenario, but it is the first replanning result recorded in Gazebo rather than in the planning code alone.

**Live fusion capture.** `scripts/capture_sensor_fusion_evidence.py` recorded a live frame, LiDAR scan and telemetry from the running system. It confirms that the pipeline runs live (10 classes loaded on `cuda:0`, 9.74 ms inference, 51.4 Hz effective detection rate), but the robot was stationary 0.38 m from a wall and the file records no fused obstacle and no sign, so it does not measure fusion accuracy.

### 5.1.17 V2.6 Ten-Mission Benchmark

The ten benchmark missions were re-run with the V2.6 configuration, with a 180 s limit for every mission (Table 5.13). These results were supplied by the project team from the V2.6 run; the raw results file is not yet committed to the repository. The earlier V2.6 figure of 10 of 10, whose values were typed into `scripts/compile_final_benchmark_results.py`, is superseded by this run and is not used.

**Table 5.13** V2.6 ten-mission benchmark (180 s limit per mission)

| Mission | Start → goal         | Result | Duration (s) | Distance (m) | Avg speed (m/s) | Collisions | Reason         |
|---------|----------------------|--------|--------------|--------------|-----------------|------------|----------------|
| M1      | Room A → Storage     | Pass   | 28.05        | 8.86         | 0.316           | 0          | Goal reached   |
| M2      | Room A → Hospital    | Fail   | 180.27       | 33.48        | 0.186           | 0          | Timeout        |
| M3      | Loading → Charging   | Fail   | 180.17       | 55.11        | 0.306           | 0          | Timeout        |
| M4      | Warehouse → Exit     | Pass   | 127.44       | 51.71        | 0.406           | 0          | Goal reached   |
| M5      | Storage → Office     | Fail   | 180.03       | 43.03        | 0.239           | 0          | Timeout        |
| M6      | Hospital → Storage   | Pass   | 86.75        | 24.20        | 0.279           | 0          | Goal reached   |
| M7      | Charging → Exit      | Pass   | 78.10        | 27.89        | 0.357           | 0          | Goal reached   |
| M8      | Warehouse → Hospital | Fail   | 180.15       | 59.13        | 0.328           | 0          | Timeout        |
| M9      | Room B → Storage     | Fail   | 180.04       | 9.27         | 0.051           | 0          | Stuck in place |
| M10     | Reception → Room A   | Pass   | 142.19       | 36.22        | 0.255           | 0          | Goal reached   |
| Total   |                      | 5 / 10 | 1363.19      | 348.90       | 0.256           | 0          |                |

Five of the ten missions reached their goal, against 0 of 10 for v2.5 and 2 of 10 for the final v2.4 run, with no collisions. The robot travelled 348.90 m in total at

an effective 0.256 m/s, about twice the v2.5 speed (0.125 m/s). Four missions timed out while still moving (M2, M3, M5 and M8, at 0.19–0.33 m/s), which points to route length and speed rather than a blockage. Mission 9 stayed stuck near its start in the ward (9.27 m in 180 s), although mission 6, which now starts from the same ward position, reached its goal. Replans, TTC yields and minimum clearance are not included in this summary.

### 5.1.18 V3.0: Hospital Logistics Extension

V3.0 (tag v3.0, commit a8ac484) moves the system into a realistic hospital and gives it a logistics task. The building is the AWS RoboMaker Hospital World with 43 Gazebo Fuel models (staff, patients, beds, wheelchairs, IV stands), a single floor of  $25 \times 56$  m mapped as a 0.05 m grid of  $27.1 \times 59.0$  m, with a 33-node, 37-edge route graph (Fig. 5.3). In each mission the robot drives to a collection point in the West Ward,  $(-8.5, -15.6)$ , collects a blood-sample carrier and delivers it to the laboratory,  $(8.6, 14.6)$ . The robot, the Nav2 parameters and the V2.6 decision engine are reused; an adapter makes the engine ignore non-moving obstacles that lie on occupied cells of the static map, which cut the first mission from 7 replans and 154 m to 0 replans and 107 m. Four traffic models (two visitors, a nurse and a trolley) cross the corridors.



**Fig. 5.3.** The robot in the main lobby of the V3.0 hospital, with the reception desk, visitors and two directional signs.

**Detector on rendered images.** The v2.5 training images are drawn with OpenCV rather than rendered by Gazebo. V3 therefore built a new dataset of 4,500 Gazebo renders (3,000 train, 600 validation, 500 test and 400 in ten challenge

subsets), labelled from Gazebo’s instance segmentation and checked against an independent 3D projection, with nine hospital classes. The V3 model is YOLOv8n fine-tuned from the v2.5 weights for 60 epochs. On the held-out test split, rendered along real robot routes in the earlier V3 world, the v2.5 model scores only 0.106 mAP@0.50, against 0.804 for the V3 model (Table 5.14; `V3/docs/evidence/phase2_eval/evaluation.json`). The weakest V3 classes are `cart` (AP 0.538) and `iv_stand` (0.514). The 0.899 of v2.5 in Section 5.1.2 was therefore measured on drawn images of the kind it was trained on, and does not carry over to rendered scenes.

**Table 5.14** Detector on the V3 test split of Gazebo renders (500 images, 1,504 instances, confidence 0.35)

| Metric                    | v2.5 (deployed thresholds) | V3 (fine-tuned from v2.5) |
|---------------------------|----------------------------|---------------------------|
| Precision                 | 0.589                      | 0.958                     |
| Recall                    | 0.294                      | 0.925                     |
| False positives per image | 0.618                      | 0.122                     |
| mAP@0.50                  | 0.106                      | 0.804                     |
| mAP@0.50–0.95             | 0.058                      | 0.689                     |

**Fusion on rendered scans.** V3 measured camera–LiDAR association on recorded Gazebo frames, each with the image, the LiDAR scan and the true object positions (`V3/docs/evidence/phase3_fusion/fusion_eval.json`). An association counts as correct when the fused LiDAR point lies within 0.25 m of the object. On the held-out set of 300 frames, the V2.6 association is correct for 51.4% of measurable objects and wrong for 47.7%. A new V3 association, which splits the LiDAR returns in the box into depth segments and checks them against where the box meets the floor, raises correct associations to 80.6% and lowers wrong ones to 7.2% (Table 5.15). It also marks all 468 signs and markers as visual-only, where V2.6 gave a false range for 435 of them. Accuracy falls with distance (63.7% correct beyond 4 m) and with camera miscalibration (a 0.79° pitch error raises wrong associations from 7.2% to 13.3%).

**Table 5.15** Camera–LiDAR association on recorded Gazebo frames

| Set                     | Method | Correct association | Wrong association | Median range error |
|-------------------------|--------|---------------------|-------------------|--------------------|
| Held-out (300 frames)   | V2.6   | 0.514               | 0.477             | 0.225 m            |
| Held-out (300 frames)   | V3     | 0.806               | 0.072             | 0.223 m            |
| Controlled (120 frames) | V2.6   | 0.761               | 0.239             | 0.129 m            |
| Controlled (120 frames) | V3     | 0.893               | 0.036             | 0.127 m            |

**Logistics missions.** In the new hospital, ten missions without traffic and five with traffic were run with `V3/tests/run_v3_missions.py`, each from wherever the previous one ended, through the collection point to the laboratory (Table 5.16). All fifteen were completed, with no collisions. The logged distances agree with the recorded paths to within 0.5 m, and the durations with the timestamps (Fig. 5.4). Against ground truth, the robot’s localisation error during the traffic runs was 0.12 m on average and 0.37 m at most. The detector was not retrained for this hospital: over 9,314 frames it detected every class, with mean confidence from 0.80 (person) down to 0.44 for the laboratory marker, but there is no labelled test set in this world, so its accuracy here is not measured.

**Table 5.16** V3.0 logistics missions in the hospital (collection point  $\rightarrow$  laboratory)

| Run             | Completed | Time (s), mean (range) | Distance (m), mean | Replans, mean (max) | TTC events, mean | Min. clearance |
|-----------------|-----------|------------------------|--------------------|---------------------|------------------|----------------|
| Without traffic | 10 / 10   | 289.3 (270.0–327.9)    | 124.0              | 0.9 (2)             | 0.2              | 0.264 m        |
| With traffic    | 5 / 5     | 319.6 (262.5–394.2)    | 140.0              | 1.4 (3)             | 0.8              | 0.177 m        |



**Fig. 5.4.** Recorded paths of the fifteen V3.0 missions on the hospital map, from the collection point (circle) to the laboratory (square).

These results are not directly comparable with the V2.6 benchmark. All fifteen V3 missions repeat one route, between the same two points, whereas the V2.6 benchmark has ten different start–goal pairs in a different building. In V3 the sign direction confirms the route but does not change it, false TTC on static objects during robot motion is reduced but not removed, and the Gazebo bridge crashes at startup in about one start in twelve.

## 5.2 Discussion

### 5.2.1 Interpretation of Results

Perception improved substantially once the training data matched the simulator: cart AP rose from 0.05 to 0.88 and forklift AP from 0.46 to 0.93. Wall false positives were removed in the controlled tests. The planning layer also behaves correctly when tested directly: it replans in 0.05 ms median, never enters any of the 4 dead ends, and keeps its memory across restarts.

However, these component results do not add up to working navigation. No mission reached its goal. Compared with the final v2.4 run, the robot travelled further and yielded less (TTC yields fell from 300 to 95, replans from 353 to 327, and the effective speed rose from 0.085 to 0.125 m/s), but completion fell from 2 to 0 missions out of 10. The project’s audit (Table 5.17) traces this to defects outside the detector: the time-to-collision logic treats static objects as approaching and halts the robot 95 times; temporary blockages stay marked in the database across runs; a stuck timer fires during legitimate yields; Nav2’s cost-regulated speed and the stop-and-go waypoint handoff limit the robot to about 0.15–0.25 m/s; and the ward start pose leaves only 0.5 m of clearance. A more accurate detector therefore does not by itself produce a more capable robot. V2.6 fixes these defects in the code (Section 5.1.15), and its controlled experiments confirm part of the effect: the controller now sustains 0.47 m/s on a straight corridor, and a live replanning run finishes without a cascade. They also show what remains: the ward start was still trapped in the start-pose experiment, and the ego-motion fix removed false yields in one run but not in the next. In the V2.6 benchmark the fixes raise mission success from 0 to 5 of 10 with no collisions, while four missions still time out and mission 9 stays stuck in the ward (Section 5.1.17).

Several inconsistencies in the source material are worth noting. First, the training report claims that background false positives fell from 82 to 2, but the metrics file records 88 background false positives on the validation split for v2.5 (46 on test);

the difference is presumably the confidence threshold used, which is not documented. Second, the sign results differ between the live test (33/33) and the controlled test (58%). Third, the V2.6 validation report states a detector mAP@0.50 of 0.942, while the metrics file gives 0.899 and the training curve 0.922, and it counts 33 physical signs in the facility, while the world file places 16 signs (using 12 of the 33 sign textures). This report uses the values in the results files.

V3.0 adds two findings that qualify the results above (Section 5.1.18). First, the v2.5 detector does not transfer from drawn images to Gazebo renders (0.106 against 0.899 mAP@0.50), so a detector for the simulated robot has to be trained on rendered images, as the V3 model is (0.804). Second, camera–LiDAR association measured on rendered frames is correct for only half the objects with the V2.6 method, which agrees with the V2.6 live capture that recorded no fused obstacle; the V3 method raises this to 81%. With both changes and a new building, the robot completes all fifteen logistics missions on one repeated route.

## 5.2.2 Significance of Findings

The results show that a semantic layer can be added above Nav2 without compromising its safety: across 230.8 m of travel in the ten-mission run there were no collisions. They also show that measuring on a leak-free split with negative samples changes the picture considerably: a detector that appeared to score 0.91 scored 0.63 when tested properly.

## 5.2.3 Limitations

Table 5.17 lists the technical limitations identified in the project’s audit of the v2.5 code and results, with their evidence and effect. The first eight were traced to a specific cause; the last is a suspected cause. V2.6 changes the code for the first seven (Section 5.1.15); the benchmark timeouts are unchanged, and the effect of the fixes on the full benchmark is not yet measured.

**Table 5.17** Technical limitations of v2.5 identified in the project’s audit

| Area            | Limitation                              | Evidence and cause                                                                                    | Effect                                            |
|-----------------|-----------------------------------------|-------------------------------------------------------------------------------------------------------|---------------------------------------------------|
| TTC             | Yields on static objects                | Closing-speed test ( $\dot{r} < -0.05$ ) in the fusion node has no check that the object is dynamic   | Each false yield halts the robot 1.8 s; 95 yields |
| TTC             | Robot’s own motion not removed          | Object velocities are computed in <code>base_link</code> without subtracting odometry                 | Static objects appear to approach                 |
| Memory          | Blockages persist across runs           | 30 edges left marked blocked in the database; hospital-to-storage route cost 4,000,175                | Optimal corridors are avoided                     |
| Decision Engine | Stuck timer fires during yields         | The 5.0 s stuck check does not pause during a TTC pause (about $2 \times 1.8$ s)                      | Spurious reverse recovery and goal abort          |
| Nav2 control    | Speed throttled in corridors            | Cost-regulated scaling gives about 0.15–0.25 m/s in 1.4–1.8 m corridors                               | Timeouts                                          |
| Nav2 control    | Stop-and-go at waypoints                | Waypoints 2–4 m apart are sent one by one; the robot slows to 0.05 m/s at each                        | No continuous cruising                            |
| Facility        | Start pose trapped in the ward (M6, M9) | Start (6.5, −9.5) is 0.5 m from a bed, facing away from the door; inflation radius 0.38 m             | 40–52 replans per run                             |
| Benchmark       | Unequal timeouts                        | 220 s for M10, 180 s for M1–M9                                                                        | Missions not comparable                           |
| Perception      | Sign recall at range                    | Recall 85.3% (precision 98.8%) beyond 5 m or at steep angles; suspected cause: boxes under 20 px wide | Low impact, temporal gate helps                   |

The wider limitations of the evidence, in order of consequence:

1. **Half of the missions complete.** The v2.5 benchmark succeeds on 0 of 10 missions and the V2.6 benchmark on 5 of 10 (Section 5.1.17), so the system’s practical navigation ability is only partly demonstrated. The causes of the v2.5 result are in Table 5.17.

2. **Cart, forklift and near-sign detection is weak in controlled tests.** Cart and forklift recall is 73–77% and overall directional-sign recall is 58%, which is below the level needed for dependable obstacle avoidance and sign following.
3. **The detector is trained and validated on synthetic renders.** The v2.5 images reproduce the Gazebo assets, but they are still generated by a script; the ten-mission run is the only test on the live simulator camera, and no result has been checked on a physical camera.
4. **Sign and fusion accuracy come partly from constructed inputs.** Scenario 1 matches each template against a pasted copy of itself, and the fusion test recovers a distance from a scan it generated with known noise.
5. **Post-processing thresholds are tuned by hand,** for example 0.75 for `hospital_bed` and the class-agnostic 60% area filter, which could reject a legitimate large object viewed at close range.
6. **The navigation ablation is a 2D model** with 5 scenarios, so it cannot rank the three modalities on safety.
7. **The topology is hand-authored.** The 48-node graph is written by hand; SLAM mode maps a new building’s geometry but not its topology.
8. **Simulation only.** Wheel slip, sensor noise and lighting would all differ on hardware.

## 5.3 Summary of Results

- Detection (v2.5, leak-free split): 0.899 mAP@0.50 on both validation and test, up from 0.634 for v2.4 on the same split; 4.06 ms per image.
- Data audit: the earlier 0.910 (v2.4) was inflated by 35 cross-split duplicate pairs and is not used.
- Controlled tests: no false positives on empty hallways, walls and shadows; recall 90% (bed), 73% (cart), 77% (forklift), 100% (person), 58% (signs).
- Planning: 0.05 ms median replan and 0 dead-end entries (re-run and verified).
- Ten-mission Gazebo benchmark (v2.5): 0 of 10 missions completed (final v2.4: 2 of 10), 0 collisions, 230.8 m travelled, 327 replans, 95 TTC yields (final v2.4: 300).

- Audit: nine limitations identified, eight with a confirmed cause (false TTC yields, blockages that persist in the database, a stuck timer that fires during yields, speed throttling, stop-and-go waypoints, a trapped start pose, unequal timeouts).
- Perception improved at unchanged speed (3.94 against 4.06 ms per image), but that did not translate into mission success.
- V3.0: in a realistic hospital, 10 of 10 logistics missions without traffic and 5 of 5 with traffic are completed with 0 collisions, on one repeated route; on Gazebo renders the V3 detector reaches 0.804 mAP@0.50 against 0.106 for v2.5, and correct camera–LiDAR association rises from 0.514 to 0.806.
- V2.6: the audited defects are fixed in the code; the controller sustains 0.472 m/s on a straight corridor and a live replanning run passes, but the ego-motion test failed once in two runs. The V2.6 benchmark completes 5 of 10 missions with 0 collisions, 348.9 m travelled at 0.256 m/s.

# Chapter 6

## Work to be Done

### 6.1 Conclusion

The project shows that a semantic layer can sit on top of Nav2 without compromising it: Nav2 keeps full responsibility for collision-free motion, while a SQLite-backed topological graph and a Decision Engine add destination-by-name routing, persistent memory, dead-end avoidance, and dynamic replanning. Across 230.84 m of simulated travel in the ten-mission benchmark there were no collisions and no manual interventions.

Two parts of the system are well supported. The planning layer, tested directly, replans in 0.05 ms median, never enters any of the 4 dead ends across 7 destinations, and keeps its memory across restarts. The perception layer was rebuilt in v2.5 on a leak-free dataset and reaches 0.899 mAP@0.50 (against 0.634 for v2.4 on the same split) at 4.06 ms per image.

The integrated system does not yet complete its task reliably: none of the ten v2.5 missions reaches its goal before the timeout, against 2 of 10 for the final v2.4 run. The audit in Chapter 5 traced this mainly to software defects and tuning outside the detector. V2.6 fixes these defects in the code, and its experiments confirm part of the effect: the controller sustains 0.472 m/s on a straight corridor and a live replanning run finishes without a cascade. In the V2.6 benchmark the robot completes 5 of the 10 missions with no collisions; four missions still time out and mission 9 stays stuck in the ward. The project’s fourteen objectives are therefore mostly met at component level, and objective 1 (navigating between destinations) is only partly met.

V3.0 carries the system into a realistic hospital with a logistics task. Retrained on Gazebo renders and with a new camera–LiDAR association, it completes all fifteen collection-to-laboratory missions, five of them with moving traffic, with no collisions. These missions repeat one route, so V3.0 shows the system working end to end in

one realistic setting rather than across a full benchmark.

## 6.2 Work to be Done

The five V2.6 targets set by the audit have been addressed as follows:

1. **Ego-motion-compensated TTC.** Implemented: the closing rate is corrected for the robot's forward speed, and yields are restricted to dynamic obstacles. The runtime test passed in one run and failed once in the next, so the remaining false yield must be traced.
2. **Session memory reset.** Implemented: transient blocked-edge flags are cleared when a mission starts, and the stuck timer is suspended during a TTC pause. The effect has not been measured separately.
3. **Local inflation tuning.** Implemented with a radius of 0.30 m, chosen from a sweep of four radii (0.28 m was the original target). The ward start pose was moved, but the robot still does not leave the ward at any tested pose or radius.
4. **Velocity control.** Implemented: the minimum regulated speed is 0.15 m/s (0.20 m/s was the original target) and waypoints are handed over without a full stop. The straight-corridor speed is confirmed; the effect of the handoff is not yet measured.
5. **Benchmark standardization.** Partly done: the V2.6 benchmark uses a 180 s limit for every mission, but the v2.4 baseline has not been re-run with the V2.6 script.

The work still needed is, in order of importance:

1. **Run a multi-route benchmark in the V3.0 hospital,** with different start and goal pairs, and build a labelled test set from that hospital to measure the detector there.
2. **Raise the V2.6 success rate above 5 of 10.** Commit the raw results file of the V2.6 benchmark with its replans, TTC yields and clearance, record trajectories from odometry, and find why missions 2, 3, 5 and 8 run out of time while still moving.
3. **Free the ward start.** Mission 6 now leaves the ward, but mission 9 stays stuck from the same start position; find the cause and re-test both.

4. **Measure the remaining fixes**, in particular the waypoint handoff and the blockage reset, with a before-and-after run each.
5. **Validate fusion on live returns**, with a moving robot and a labelled obstacle at known distances.
6. **Improve perception**: directional-sign recall (especially at near range and at oblique angles) and cart and forklift recall, explain the difference between the live sign test (33/33) and the controlled test (58%), and evaluate the detector on the robot's live camera and eventually on a physical robot (for example TurtleBot 4) with 3D LiDAR for multi-height obstacles.

## References

- C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard. Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age. *IEEE Transactions on Robotics*, 32(6):1309–1332, 2016. doi: 10.1109/TRO.2016.2624754.
- E. Garcia-Fidalgo and A. Ortiz. Vision-based topological mapping and localization methods: A survey. *Robotics and Autonomous Systems*, 64, 2015.
- P. E. Hart, N. J. Nilsson, and B. Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>, 2023.
- N. Koenig and A. Howard. Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2149–2154, 2004.
- I. Kostavelis and A. Gasteratos. Semantic mapping for mobile robotics tasks: A survey. *Robotics and Autonomous Systems*, 66:86–103, 2015.
- J. P. Lewis. Fast template matching. In *Vision Interface*, pages 120–123, 1995.
- S. Macenski, F. Martín, R. White, and J. Ginés Clavero. The marathon 2: A navigation system. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. arXiv:2003.00368.
- S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall. Robot operating system 2: Design, architecture, and uses in the wild. *Science Robotics*, 7(66): eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.
- S. Macenski, S. Singh, F. Martín, and J. Ginés. Regulated pure pursuit for robot path tracking. *Autonomous Robots*, 2023. arXiv:2305.20026.

- J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016.
- J. Terven and D.-M. Córdoba-Esparza. A comprehensive review of YOLO architectures in computer vision: From YOLOv1 to YOLOv8 and YOLO-NAS. *Machine Learning and Knowledge Extraction*, 5(4):1680–1716, 2023. doi: 10.3390/make5040083.
- J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017. doi: 10.1109/IROS.2017.8202133.
- S. Vora, A. H. Lang, B. Helou, and O. Beijbom. PointPainting: Sequential fusion for 3D object detection. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. arXiv:1911.10150.

# Annexure I

## I.1 Deployment Defects Found on a Second Machine

Bringing the workspace up from a clean clone surfaced four build/runtime defects, each traced to its root cause and fixed (Table 6.1).

**Table 6.1** Build and runtime defects found on a clean clone

| # | Symptom                                                             | Root cause                                                                                                                                               | Fix                                              |
|---|---------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| 1 | colcon build fails: cannot find autonomous_robot_description/meshes | CMakeLists.txt installs a meshes/ directory that Git never tracked, since Git does not version empty folders; no URDF/xacro file references a mesh asset | Added the missing empty directory                |
| 2 | Same failure for autonomous_robot_gazebo/models                     | Identical empty-directory issue; the world SDF files define all models inline with no external references                                                | Added the missing empty directory                |
| 3 | Same failure for autonomous_robot_bringup/config                    | Same root cause, found by auditing every install(DIRECTORY ...) call in all 5 packages                                                                   | Added the missing empty directory                |
| 4 | Launch fails: executable initial_pose_publisher not found           | The script exists and is referenced by localization.launch.py, but install(PROGRAMS ...) registered only two of the three executables                    | Added the missing entry to install(PROGRAMS ...) |

## I.2 Incomplete Dependency List

A fifth issue was an incomplete dependency list rather than a repository bug: the original install documentation lists individual Nav2 packages (`nav2-amcl`, `nav2-planner`, `nav2-controller`, etc.) but omits the `navigation2` and `nav2-bringup` metapackages, which supply `nav2_behaviors` and `nav2_bt_navigator`, both required at launch for the action client that dispatches `/navigate_to_pose` goals. Adding `ros-lyrical-navigation2` and `ros-lyrical-nav2-bringup` to the Docker image resolved it.

## I.3 Hardcoded Paths

The sixth defect was the most serious because it failed silently. Launch-file defaults, node parameters, and 19 scripts all hardcoded the original developer’s home directory. On any other machine the simulation still started and Nav2 still drove, but `object_detection_node` crashed on startup looking for YOLO weights at that path, and `sign_detection_node` loaded zero templates and quietly detected nothing, so the headline perception features were off while the system appeared to run. The test suite could not find the project either. Paths now resolve from a `CV_NAV_ROOT` variable that `start_project.sh` exports, falling back to each file’s own location; scenarios 2–4 were re-run from a different checkout path to confirm.

## I.4 Status of the Fixes

All six fixes were pushed to a fork for reproducibility. The five repository defects (the three missing directories, the unregistered executable, and the hardcoded paths) are candidates for a pull request back upstream, since they break any fresh clone regardless of host OS.

## I.5 Attribution

Original software and navigation modules are released under the Apache-2.0 License. Third-party simulation assets (AWS RoboMaker hospital/warehouse models, Gazebo Fuel assets) and library attributions are documented in the repository’s `docs/THIRD_PARTY.md`. The project is built on ROS 2 Lyrical, Gazebo Sim 10.5, Nav2, and Ultralytics YOLOv8.

# Annexure II

## II.1 Object Inventory

Table 6.2 lists the objects in the simulated facility (`realistic_facility_world.sdf`), as recorded in the project’s own object inventory. The “YOLO class” column gives the class the inventory associates with each object; the entries marked “obstacle” are the inventory’s assumption and were not evaluated separately in the detector tests of Chapter 5. Quantities are approximate.

**Table 6.2** Object inventory of the simulated facility

| Object                   | S/D | YOLO class | Role in the world                                                                                                                                           | Qty |
|--------------------------|-----|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Dynamic warehouse cart   | D   | cart       | Patrols along the Y axis<br>( $X = 8.0$ , $Y \in [3.5, 5.25]$ )<br>at 0.35 m/s; triggers TTC<br>yields and corridor<br>replanning                           | 1   |
| Dynamic hospital trolley | D   | cart       | Patrols along the X axis<br>( $Y = -6.5$ , $X \in [5.0, 7.0]$ )<br>at 0.25 m/s across the<br>hospital ward doorway;<br>causes doorway clearance<br>blockage | 1   |
| Autonomous forklift      | D   | forklift   | Patrols the loading bay /<br>warehouse junction<br>( $X = 11.5$ ,<br>$Y \in [4.5, 5.82]$ ) at<br>0.22 m/s; largest safety<br>buffer (1.10 m radius)         | 1   |

| Object                                                     | S/D | YOLO class       | Role in the world                                                                                                                                       | Qty |
|------------------------------------------------------------|-----|------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----|
| Walking personnel                                          | D   | person           | Moves along the cafeteria corridor ( $X = -6.0$ , $Y \approx -6.5$ ) at 0.30 m/s; 0.90 m safety radius                                                  | 1   |
| Hospital beds (hospital_bed_1, _2)                         | S   | hospital_bed     | Inpatient beds ( $2.0 \times 1.0 \times 0.7$ m) in the hospital ward; narrow-clearance constraint (0.5 m) at the M6/M9 starting poses                   | 2   |
| Pallet stacks (wh_pallets, loading_pallet_stack)           | S   | pallet           | Industrial pallet stacks ( $1.2 \times 1.2 \times 0.8$ m) marking the loading bay dock and the warehouse perimeter                                      | 2   |
| Storage box stack                                          | S   | box              | Stacked cartons ( $1.2 \times 1.0 \times 1.0$ m) along the western wall of the storage depot                                                            | 1   |
| Charging dock station                                      | S   | charging_station | Automated AGV charging terminal ( $0.6 \times 1.0 \times 1.0$ m) on the western outer wall; goal of missions M3 and M7                                  | 1   |
| Directional signboards (sign_j1_* to sign_j8_*, sign_j6_*) | S   | directional_sign | High-mounted boards ( $0.8 \times 0.4$ m at $z = 1.10$ m) with textures; guide topological route selection and trigger closed-edge rerouting            | 16  |
| Corridor doorways and openings                             | S   | door             | Passage openings ( $1.40\text{--}1.60$ m clear width) connecting rooms and corridors, including the emergency exit threshold (exit at $(-12.5, -9.0)$ ) | 9   |

| Object                                              | S/D | YOLO class | Role in the world                                                                                                      | Qty |
|-----------------------------------------------------|-----|------------|------------------------------------------------------------------------------------------------------------------------|-----|
| Warehouse storage racks<br>(wh_rack_1 to wh_rack_6) | S   | obstacle   | Multi-tier metal shelving ( $0.8 \times 2.4 \times 2.0$ m) defining warehouse aisles at $X = 11.5$ and $X = 14.0$      | 6   |
| Workstations and lab benches                        | S   | obstacle   | Heavy workbenches ( $2.0\text{--}2.4$ m long) in Room A, the lab and the warehouse hub                                 | 4   |
| Office desks and meeting tables                     | S   | obstacle   | Desks ( $1.5 \times 0.8$ m) in the office suite and dining tables ( $1.4 \times 0.9$ m) in the cafeteria               | 4   |
| Bedside tables                                      | S   | obstacle   | Utility side tables ( $0.5 \times 0.5 \times 0.7$ m) next to the hospital beds                                         | 2   |
| Chairs and benches                                  | S   | obstacle   | Office and cafeteria chairs and reception waiting benches ( $1.6$ m long)                                              | 6   |
| Reception desk                                      | S   | obstacle   | Facility check-in counter ( $1.8 \times 0.6 \times 0.9$ m) in the southern entrance foyer                              | 1   |
| Perimeter and partition walls<br>(wall_*)           | S   | none       | Structural partitions ( $2.8$ m high, $0.2$ m thick); mapped in the 2D costmap and rejected by the geometric filter    | 37  |
| Support pillars<br>(pillar_*)                       | S   | none       | Square concrete columns ( $0.5 \times 0.5 \times 2.8$ m) in reception, the central crossway, the warehouse and the lab | 6   |
| Ground plane                                        | S   | none       | Textured floor ( $36 \times 30$ m) at $z = 0$                                                                          | 1   |
| Ceiling lights and sun                              | S   | none       | One directional sun plus 10 ceiling point lights ( $3000\text{--}5500$ K)                                              | 11  |

## II.2 Summary

The world contains 93 SDF models: 4 dynamic agents (controlled by `dynamic_obstacles_node.py`), 29 items of static furniture and equipment, 16 directional signs registered in `semantic_map.yaml`, and 43 static walls and pillars mapped into the 0.05 m occupancy grid (`realistic_facility_map.yaml`). All 10 trained classes appear in the world.

## II.3 Semantic Destinations

Table 6.3 lists the eleven named destinations of the facility (from `semantic_map.yaml`) and the benchmark missions that start or end at each. The hospital and Room B share the same coordinates.

**Table 6.3** Semantic destinations and their use in the ten-mission benchmark

| Destination     | Zone                            | Position (X, Y) in m           | Junction          | Missions                  |
|-----------------|---------------------------------|--------------------------------|-------------------|---------------------------|
| Reception       | Main reception foyer            | (0.0, -8.6)                    | junction_1        | start M10                 |
| Hospital        | Hospital medical ward           | (6.5, -9.5); V2.6: (5.0, -9.0) | junction_2        | goal M2, M8; start M6     |
| Room B          | Medical inspection room         | (6.5, -9.5); V2.6: (5.0, -9.0) | junction_2        | start M9                  |
| Storage         | Storage depot                   | (-6.5, 3.0)                    | junction_7        | goal M1, M6, M9; start M5 |
| Room A          | Research suite                  | (-7.0, 9.5)                    | junction_8        | start M1, M2; goal M10    |
| Loading         | Cargo loading bay               | (12.8, 2.0)                    | wh_junction_6     | start M3                  |
| Charging        | AGV charging station            | (-13.5, 2.0)                   | west_bypass_north | goal M3; start M7         |
| Warehouse       | Warehouse hub                   | (8.0, 7.0)                     | wh_junction_6     | start M4, M8              |
| Exit            | Emergency exit door             | (-12.5, -9.0)                  | junction_3        | goal M4, M7               |
| Office          | Executive office suite          | (0.0, 10.0)                    | junction_5        | goal M5                   |
| Utility service | Utility service room (dead end) | (-15.0, -4.5)                  | junction_3        | none                      |

# Appendix A

## Similarity Index Report

Appendix A contains the attachment of the following particulars:

- Similarity Index Report (first page)
- AI Similarity Report (first page)